

CHAPTER 9 Recursion

Recursion occurs when a method calls itself. The recursion can be direct (when the method calls itself) or indirect (when the method calls some other method that then calls the first method).

Recursion can also be single (when the method calls itself once) or multiple (when the method calls itself multiple times).

Recursive algorithms can be confusing because people don't naturally think recursively. For example, to paint a fence, you probably would start at one end and start painting until you reach the other end. It is less intuitive to think about breaking the fence into left and right halves and then solving the problem by recursively painting each half.

However, some problems are naturally recursive. They have a structure that allows a recursive algorithm to easily keep track of its progress and find a solution. For example, trees are naturally recursive because branches divide into smaller branches that divide into still smaller branches and so on. For that reason, algorithms that build, draw, and search trees are often recursive.

This chapter explains some useful algorithms that are naturally recursive. Some of these algorithms are useful by themselves, but learning how to use recursion in general is far more important than learning how to solve a single problem. Once you understand recursion, you can find it in many programming situations.

Recursion is not always the best solution, however, so this chapter also explains how you can remove recursion from a program when recursion might cause poor performance.

Basic Algorithms

Some problems have naturally recursive solutions. The following sections describe several naturally recursive algorithms for calculating factorials, finding Fibonacci numbers, solving the rod-cutting problem, and moving disks in the Tower of Hanoi puzzle.

These relatively straightforward algorithms demonstrate important concepts used by recursive algorithms. Once you understand them, you'll be ready to move on to the more complicated algorithms described in the rest of this chapter.

Factorial

The factorial of a number N is written $N!$ and pronounced "N factorial." You can define the `factorial` function recursively as follows:

$$\begin{aligned}0! &= 1 \\ N! &= N \times (N - 1)!\end{aligned}$$

For example, the following equations show how you can use this definition to calculate $3!$:

$$3! = 3 \times 2! = 3 \times 2 \times 1! = 3 \times 2 \times 1 \times 0! = 3 \times 2 \times 1 \times 1$$

This definition leads to the following simple recursive algorithm:

```
Integer: Factorial(Integer: n)
    If (n == 0) Then Return 1
    Return n * Factorial(n - 1)
End Factorial
```

First, if the input value `n` equals 0, the algorithm returns 1. This corresponds to the first equation that defines the `factorial` function. That equation is called the *base case*. It represents the situation where the algorithm does not call itself recursively.

NOTE It is important that any recursive algorithm have a base case to prevent it from recursively calling itself forever.

Otherwise, if the input is not 0, the algorithm returns the number n times the factorial of $n - 1$. This step corresponds to the second equation that defines the `factorial` function.

This is a simple algorithm, but it demonstrates two important features that all recursive algorithms must have.

- Each time the method executes, it reduces the current problem to a smaller instance of the same problem and then calls itself to solve the smaller problem. In this example, the method reduces the problem of computing $n!$ to the problem of computing $(n - 1)!$ and then multiplying by n .
- The recursion must eventually stop. In this example, the input parameter n decreases with each recursive call until it equals 0. At that point, the algorithm reaches its base case, returns 1, and does not call itself recursively, so the process stops.

Note that even this simple algorithm can create problems. If a program calls the `Factorial` method, passing it the parameter -1 , the recursion never ends. Instead, the algorithm begins the following series of calculations:

$$\begin{aligned} -1! &= \\ -1 \times -2! &= \\ -1 \times -2 \times -3! &= \\ -1 \times -2 \times -3 \times -4! &= \dots \end{aligned}$$

One method that some programmers use to prevent this is to change the first statement in the algorithm to `If (n <= 0) Then Return 1`. Now if the algorithm is called with a negative parameter, it simply returns 1.

NOTE From a software engineering point of view, this may not be the best solution because it hides a problem in the program that called the algorithm. It also returns the misleading value 1 when the true factorial of a negative number is undefined.

To detect problems in the calling code quickly, it may be better to check the value explicitly to make sure that it is at least 0 and raise an error or throw an exception if it is not.

Analyzing the run-time performance of recursive algorithms is sometimes tricky, but it is easy for this particular algorithm. On input N , the `Factorial` algorithm calls itself $N + 1$ times to evaluate $N!, (N - 1)!, (N - 2)!, \dots, 0!$. Each call to the algorithm does a small constant amount of work, so the total run time is $O(N)$.

Because the algorithm calls itself $N + 1$ times, the maximum depth of recursion is also $O(N)$. In some programming environments, the maximum possible depth of recursion may be limited, so this might cause a problem.

SERIOUS STACK SPACE

Normally, a computer allocates two areas of memory for a program: the stack and the heap.

The stack is used to store information about method calls. When a piece of code calls a method, information about the call is placed on the stack. When the method returns, that information is popped off the stack, so the program can resume execution just after the point where it called the method. (The stack is the same kind of stack described in Chapter 5.) The list of methods that were called to get to a particular point of execution is called the *call stack*.

The heap is another piece of memory that the program can use to create variables and perform calculations.

Typically, the stack is much smaller than the heap. The stack usually is large enough for normal programs because your code typically doesn't include methods calling other methods to a very great depth. However, recursive algorithms can sometimes create extremely deep call stacks and exhaust the stack space, causing the program to crash.

For this reason, it's important to evaluate the maximum depth of recursion that a recursive algorithm requires in addition to studying its run time and memory requirements.

The `factorial` function grows very quickly, so there's a practical limit to how big N can be in a normal program. For example, $20! \approx 2.4 \times 10^{18}$, and $21!$ is too big to fit in a 64-bit long integer. If a program never calculates values larger than $20!$, the depth of recursion can be only 20 and there should be no problem with stack space.

If you really need to calculate larger factorials, you can use other data types that can hold even larger values. For example, a 64-bit double-precision floating-point number can hold $170! \approx 7.3 \times 10^{306}$. Some data types, such as the `.NET BigInteger` type and Python's integer types, can hold arbitrarily large numbers. In those cases, the maximum depth of recursion could be a problem. The section "Tail Recursion Removal" later in this chapter explains how you can prevent this kind of deep recursion from exhausting the stack space and crashing the program.

Fibonacci Numbers

The Fibonacci numbers (named after the Italian mathematician Leonardo of Pisa, c. 1170–c. 1250, who was later known as Fibonacci) are defined by these equations:

$$\text{Fibonacci}(0) = 0$$

$$\text{Fibonacci}(1) = 1$$

$$\text{Fibonacci}(n) = \text{Fibonacci}(n - 1) + \text{Fibonacci}(n - 2) \text{ for } n > 1$$

To calculate a new Fibonacci number, you add the two preceding values. For example, the first 12 Fibonacci numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89.

NOTE Some people define $\text{Fibonacci}(0) = 1$ and $\text{Fibonacci}(1) = 1$. This gives the same values as the definition shown here, just skipping the value 0.

The recursive definition leads to the following recursive algorithm:

```
Integer: Fibonacci(Integer: n)
    If (n <= 1) Then Return n
    Return Fibonacci(n - 1) + Fibonacci(n - 2);
End Fibonacci
```

If the input n is 0 or 1, the algorithm returns 0 or 1. (If the input is 1 or less, the algorithm just returns the input.)

Otherwise, if the input is greater than 1, the algorithm calls itself for inputs $n - 1$ and $n - 2$, adds them together, and returns the result.

This recursive algorithm is reasonably easy to understand but is very slow. For example, to calculate $\text{Fibonacci}(6)$ the program must calculate $\text{Fibonacci}(5)$ and $\text{Fibonacci}(4)$. But before it can calculate $\text{Fibonacci}(5)$, the program must calculate $\text{Fibonacci}(4)$ and $\text{Fibonacci}(3)$. Here $\text{Fibonacci}(4)$ is being calculated twice. As the recursion continues, the same values must be calculated many times. For large values of N , $\text{Fibonacci}(N)$ calculates the same values an enormous number of times, making the program take a very long time.

Figure 9.1 shows the Fibonacci algorithm's call tree when it evaluates $\text{Fibonacci}(6)$. Each node in the tree represents a call to the algorithm with the indicated number as a parameter. The figure shows, for example, that the call to $\text{Fibonacci}(6)$ in the top node calls $\text{Fibonacci}(5)$ and $\text{Fibonacci}(4)$. If you look at the figure, you can see that the tree is filled with duplicated calls. For example, $\text{Fibonacci}(0)$ is calculated five times, and $\text{Fibonacci}(1)$ is calculated eight times.

This algorithm's run time is more complicated than that of the Factorial algorithm because this algorithm is multiply-recursive. To understand the run time, suppose $T(N)$ is the run time for the algorithm on input N . If $N > 1$, the algorithm calculates $\text{Fibonacci}(N - 1)$ and $\text{Fibonacci}(N - 2)$, performs an extra step to add those values, and returns the result. That means $T(N) = T(N - 1) + T(N - 2) + 1$.

This is slightly greater than $T(N - 1) + T(N - 2)$. If you ignore the extra constant 1 at the end, this is the same as the definition of the Fibonacci function, so the algorithm has a run time at least as large as the function itself.

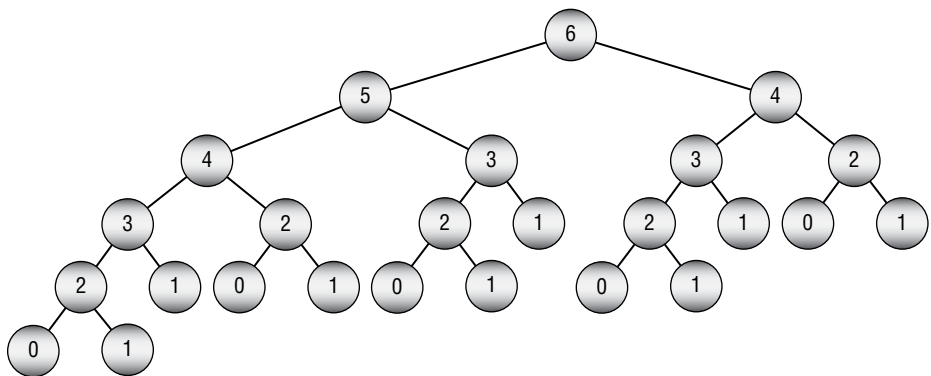


Figure 9.1: The Fibonacci algorithm’s call tree is filled with duplicated calculations.

The `Fibonacci` function doesn’t grow as quickly as the `Factorial` function, but it still grows very quickly. For example, $\text{Fibonacci}(92) \approx 7.5 \times 10^{18}$ and $\text{Fibonacci}(93)$ doesn’t fit in a long integer. That means you can calculate up to $\text{Fibonacci}(92)$ with a maximum depth of recursion of 92, which shouldn’t be a problem for most programming environments.

However, the run time of the Fibonacci algorithm grows quickly. On my computer, calculating $\text{Fibonacci}(44)$ takes more than a minute, so calculating much larger values of the function is impractical anyway. (Don’t worry. All is not lost. Techniques later in this chapter explain how you can calculate much larger Fibonacci numbers.)

Rod-Cutting

In the *rod-cutting problem*, the goal is to cut a wooden or metal rod into pieces. Pieces of different lengths have different values, so you must decide the best way to cut the rod to maximize the total value.

NOTE The rod-cutting problem assumes that the rod and pieces all have integer lengths. I’ll also assume that the piece values are integers, although you could use floating-point values without changing the solution.

For example, suppose the following table shows the values of pieces with different lengths:

Length:	1	2	3	4	5	6	7	8
Value:	1	5	8	9	10	17	17	20

Now suppose that the original rod is 10 inches long. You could cut it into ten 1-inch pieces. Each 1-inch piece has a value of 1, so the divided rod would have a total value of $10 \times 1 = 10$.

Another solution would be to divide the rod into two 5-inch pieces. Each of those would have a value of 10, so the divided rod would have a total value of $2 \times 10 = 20$.

There are many other ways that you could divide the rod. You could divide the rod into pieces with lengths $1 + 2 + 7$, $3 + 3 + 4$, $4 + 6$, or any other combination of lengths that add up to 10.

The following sections describe three approaches that you can take to find the best combination of cuts to maximize the total value.

Brute Force

In the *brute-force* approach, you simply examine every possible combination of cuts, evaluate their values, and pick the one that gives the best result. If the rod initially has length N , then there are $N - 1$ places where you could make cuts. At each of those places, you either make a cut or not, so there are 2^{N-1} possible combinations of cuts.

In the brute-force solution, you examine all 2^{N-1} possible cuts, evaluate their total values, and pick the best one. Because this requires you to examine 2^{N-1} possible solutions, it has a run time of $O(2^N)$.

This approach includes some duplication. For example, the cuts $1 + 1 + 8$, $1 + 8 + 1$, and $8 + 1 + 1$ all produce pieces with the same lengths. Even if you eliminate duplicates, however, the number of possible cuts is still exponential, so this is a relatively slow approach.

Recursion

There are many ways that you could enumerate all of the possible cuts to use in the brute-force approach. One method is to use recursion. Instead of trying to enumerate all possible cuts for a rod of a given length, you examine each possible single cut and then recursively find the optimal cuts for the two pieces on either side of that cut.

For example, suppose that the initial rod has length 10. You would consider the cuts $0 + 10$, $1 + 9$, $2 + 8$, $3 + 7$, $4 + 6$, and $5 + 5$. You don't need to consider making any other single cut because they give you the same results as their reverses. For example, you don't need to consider $7 + 3$, because it gives you the same result as $3 + 7$.

For each of those single cuts, you recursively consider the best way to divide the two pieces. For example, when you examine the $4 + 6$ cut, you recursively ask the method to find the optimal cuts for rods of lengths 4 and 6.

At this point, you can write an algorithm that uses recursion to find optimal cuts. Suppose `values` is an array giving the values of pieces of different lengths.

For example, `values[2]` is the value of a piece with length 2. The following pseudocode shows a recursive method for finding optimal cuts:

```
FindOptimalCuts(Integer: length, List Of Integer: values,
    Output Integer: bestValue, Output List Of Integer: bestCuts)

    // Assume we make no cuts.
    bestValue = values[length]
    bestCuts = New List Of Integer()
    bestCuts.Add(length)

    // Try possible single cuts.
    For i = 1 to length / 2
        // Try pieces with lengths i and length - i.
        Integer: value1, value2
        List Of Integer: cuts1, cuts2
        FindOptimalCuts(i, values, Output value1, Output cuts1)
        FindOptimalCuts(length - i, values,
            Output value2, Output cuts2)

        // See if this is an improvement.
        If (value1 + value2 > bestValue) Then
            bestValue = value1 + value2
            bestCuts = cuts1
            cuts1.AddRange(cuts2)
        End If
    Next i
End FindOptimalCuts
```

The algorithm first considers the case where the rod is not cut, in which case the full rod is worth `values[length]`.

Next, the algorithm loops through possible lengths for the first piece of a single cut. As I mentioned in the preceding section, the code only needs to check cuts where the first piece has length between 1 and `length / 2` because other cuts are reversed versions of those.

Each time through the loop, the algorithm recursively calls itself to find optimal cuts for the two pieces. If the sum of the two piece values is greater than the current best value, the algorithm saves the new value.

After it has examined all of the possible single cuts, the algorithm returns the greatest value that it found.

Figure 9.2 shows the algorithm's call tree when the initial rod has length 5. Notice that the calls come in pairs that evaluate the two pieces that form a single cut.

If you study the tree shown in Figure 9.2, you'll see that it contains a lot of duplication. For example, the algorithm is called twice for pieces of length 3, five times for pieces of length 2, and a whopping 14 times for pieces of length 1.

This is the same problem that spoiled the performance of the recursive Fibonacci number calculation. In fact, if you compare this tree to the branch of the tree in Figure 9.1 that calculates `Fibonacci(5)`, you'll see that the problem is even larger


```

If (n > 1) Then TowerOfHanoi(other_peg, to_peg, from_peg, n - 1)
End TowerOfHanoi

```

The first step requires faith in the algorithm that seems to beg the question of how the algorithm works. That step moves the top $n - 1$ disks from the original peg to the peg that isn't the destination peg. It does this by recursively calling the `TowerOfHanoi` algorithm. At this point, how can you know that the algorithm works and can handle the smaller problem?

The answer is that the method recursively calls itself repeatedly if needed to move smaller and smaller stacks of disks. At some point in the sequence of recursive calls, the algorithm is called to move only a single disk. Then the algorithm doesn't call itself recursively. It simply moves the disk and returns.

The key is that each recursive call is used to solve a smaller problem. Eventually, the problem size is so small that the algorithm can solve it without calling itself recursively.

As each recursive call returns, the algorithm's calling instance moves a single disk and then calls itself recursively again to move the smaller stack of disks to its final destination peg.

Figure 9.4 shows the series of high-level steps needed to move the stack from the first peg to the second. The first step recursively moves all but the bottom disk from the first peg to the third. The second step moves the bottom disk from the first peg to the second. The final step recursively moves the disks from the third peg to the second.

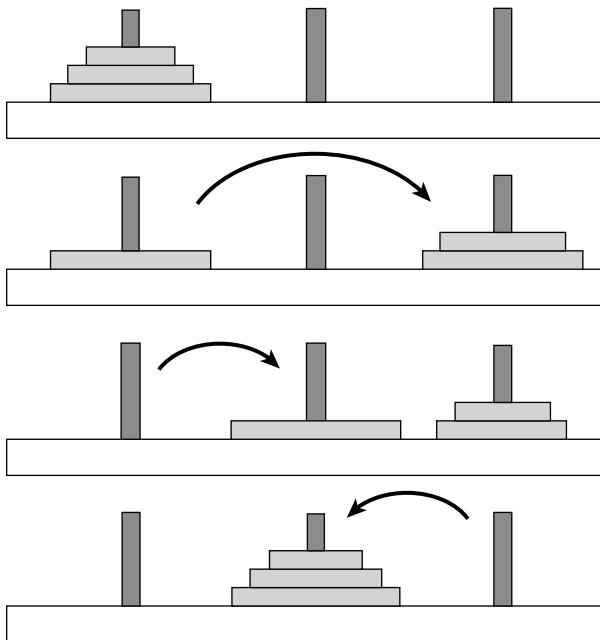


Figure 9.4: To move n disks, recursively move the upper $n - 1$ disks to the temporary peg. Then move the remaining disk to the destination peg. Finally, move the $n - 1$ upper disks from the temporary peg to the destination peg.

Figure 9.5 shows the entire sequence of moves needed to transfer a stack of three disks from the first peg to the second.

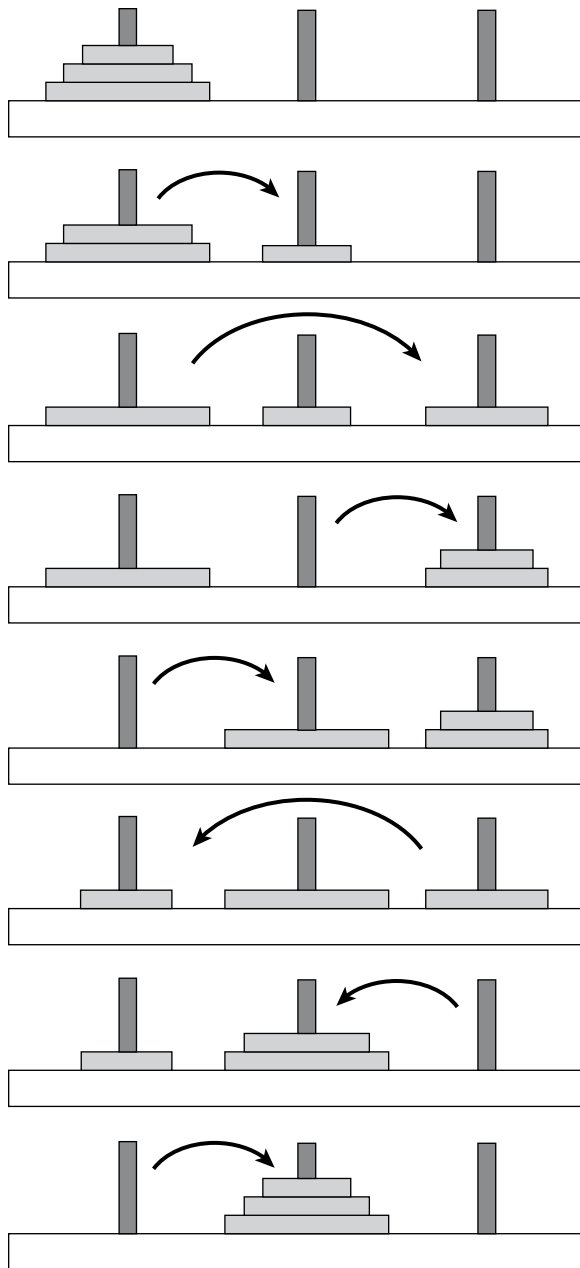


Figure 9.5: This sequence of steps transfers three disks from the first peg to the second.

To analyze the algorithm's run time, let $T(N)$ be the number of steps required to move N disks from one peg to another. Clearly $T(1) = 1$, because it takes one step to move a single disk from one peg to another.

For $N > 0$, $T(n) = T(N - 1) + 1 + T(N - 1) = 2 \times T(N - 1) + 1$. If you ignore the extra constant 1, $T(N) = 2 \times T(N - 1)$, so the function has exponential run time $O(2^N)$.

To see this in another way, you can make a table similar to Table 9.1, giving the number of steps for various values of N . In the table, each value with $N > 1$ is calculated from the previous value by using the formula $T(N) = 2 \times T(N - 1) + 1$. If you study the values, you'll see that $T(N) = 2^N - 1$.

Table 9.1: Run Times for the Tower of Hanoi Puzzle

N	T(N)
1	1
2	3
3	7
4	15
5	31
6	63
7	127
8	255
9	511
10	1023

Like the Fibonacci algorithm, the maximum depth of recursion for this algorithm on input N is N . Also like the Fibonacci algorithm, this algorithm's run time increases very quickly as N increases, so the run time limits the effective problem size long before the maximum depth of recursion does.

Graphical Algorithms

Several interesting graphical algorithms take advantage of recursion to produce intricate pictures with surprisingly little code. Although these algorithms are short, generally they are more confusing than the basic algorithms described in the previous sections.

Koch Curves

The Koch curve (named after Swedish mathematician Helge von Koch, 1870–1924) is a good example of a particular kind of *self-similar fractal*, a curve in which pieces of the curve resemble the curve as a whole. These fractals start with an *initiator*, a curve that determines the fractal's basic shape. At each level of recursion, some or all of the initiator is replaced by a suitably scaled, rotated, and translated version of a curve called the *generator*. At the next level of recursion, pieces of the generator are then similarly replaced by new versions of the generator.

The simplest Koch curve uses a line segment as an initiator. Then, at each level of recursion, the segment is replaced with four segments that are one-third of the original segment's length. The first segment is in the direction of the original segment, the next is rotated -60 degrees, the third is rotated 120 degrees, and the final segment is in the original direction. Figure 9.6 shows this curve's initiator (top) and generator (bottom).

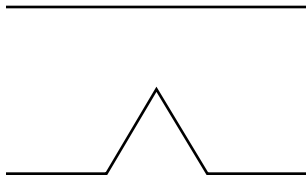


Figure 9.6: This initiator (top) and generator (bottom) create the Koch curve.

At the next level of recursion, the program takes each of the segments in the generator and replaces them with a new copy of the generator. Figure 9.7 shows the curve with levels of recursion 0 through 5.

Looking at Figure 9.7, you can see why the curve is called self-similar. Parts of the curve look like smaller versions of the whole curve.

Let `pt1`, `pt2`, `pt3`, `pt4`, and `pt5` be the points connected by the segments in the generator on the bottom in Figure 9.6. The following pseudocode shows how you might draw Koch curves:

```
// Draw a Koch curve with the given depth starting at point p1
// and going the distance "length" in the direction "angle."
DrawKoch(Integer: depth, Point: pt1, Float: angle, Float: length)
    If (depth == 0) Then
        <Draw the line segment>
    Else
        <Find the points pt2, pt3, and pt4.>
```

```

    // Recursively draw the pieces of the curve.
    DrawKoch(depth - 1, pt1, angle, length / 3);
    DrawKoch(depth - 1, pt2, angle - 60, length / 3);
    DrawKoch(depth - 1, pt3, angle + 60, length / 3);
    DrawKoch(depth - 1, pt4, angle, length / 3);
End If
End DrawKoch

```

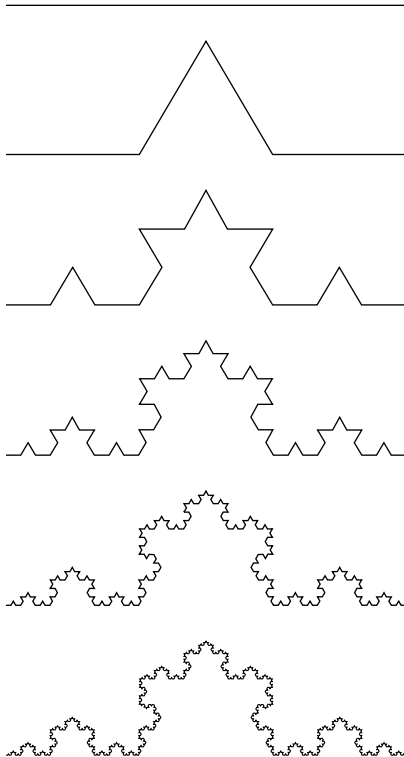


Figure 9.7: The Koch curve with levels of recursion 0 through 5 produces these shapes.

If `depth` is 0, the algorithm simply draws a line segment starting at point `p1` going in direction `angle` for the distance `length`. (Exactly how you draw the segment depends on your programming environment.)

If `depth` is greater than 0, the algorithm finds the points `pt2`, `pt3`, and `pt4`. It then draws a line segment starting at point `pt1` going in direction `angle` for one-third of the original distance. From the new end point at `pt2`, it turns 60 degrees left and draws another segment one-third of the original distance. From the new end point at `pt3`, it turns 120 degrees right (to an angle 60 degrees greater than the original angle) and draws another segment one-third of the original distance. Finally, from the last end point, `pt4`, the algorithm draws a segment at the original angle for one-third of the original distance.

If the depth is greater than 0, the algorithm calls itself recursively four times. If $T(N)$ is the number of steps the algorithm uses for depth N , $T(N) = 4 \times T(N - 1) + C$ for some constant C . If you ignore the constant, $T(N) = 4 \times T(N - 1)$, so the algorithm has run time $O(4^N)$.

The maximum depth of recursion required to draw a depth N Koch curve is only N . Like the Fibonacci and Tower of Hanoi algorithms, this algorithm's run time grows so quickly that the maximum depth of recursion should never be a problem.

If you connect the edges of three Koch curves so that their initiators form a triangle, the result is called a *Koch snowflake*. Figure 9.8 shows a level 3 Koch snowflake.

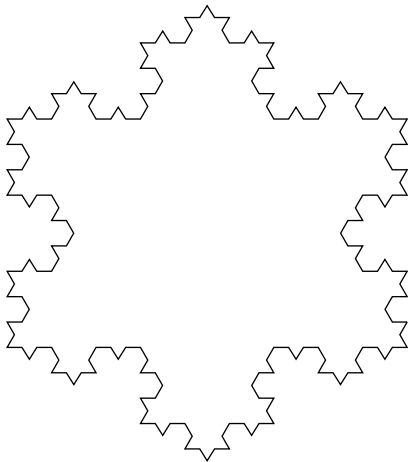


Figure 9.8: Three connected Koch curves form a Koch snowflake.

Hilbert Curve

Like the Koch curve, the Hilbert curve starts with a simple initiator curve. To move to a deeper level of recursion, the algorithm breaks the initiator into pieces and uses an appropriately rotated smaller version of the Hilbert curve to draw the pieces. (The curve was named after German mathematician David Hilbert, 1862–1943.)

Figure 9.9 shows level 0, 1, and 2 Hilbert curves. In the level 1 and 2 curves, the lines connecting the lower-level curves are gray so that you can see how the pieces are connected to build the higher-level curves.

The following pseudocode shows the remarkably simple Hilbert curve algorithm:

```
// Draw the Hilbert initially moving in the direction <dx, dy>.
Hilbert(Integer: depth, Float: dx, Float: dy)
    If (depth > 0) Then Hilbert(depth - 1, dy, dx)
```

```

DrawRelative(dx, dy)
If (depth > 0) Then Hilbert(depth - 1, dx, dy)
DrawRelative(dy, dx)
If (depth > 0) Then Hilbert(depth - 1, dx, dy)
DrawRelative(-dx, -dy)
If (depth > 0) Then Hilbert(depth - 1, -dy, -dx)
End Hilbert

```

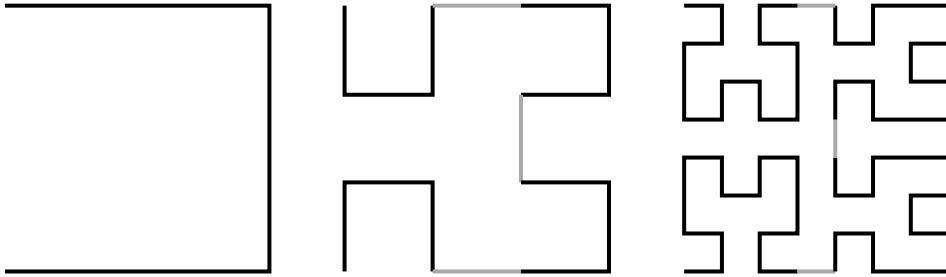


Figure 9.9: High-level Hilbert curves are made up of four connected lower-level curves.

The algorithm assumes that the program has defined a current drawing location. The `DrawRelative` method draws from that current position to a new point relative to that position and updates the current position. For example, if the current position is (10, 20), the statement `DrawRelative(0, 10)` would draw a line segment between (10, 20) and (10, 30) and leave the new current position at (10, 30).

If the depth of recursion is greater than 0, the algorithm calls itself recursively to draw a version of the curve with one lower level of recursion and with the `dx` and `dy` parameters switched so that the smaller curve is rotated 90 degrees. If you compare the level 0 and level 1 curves shown in Figure 9.9, you can see that the level 0 curve drawn at the beginning of the level 1 curve is rotated.

Next the program draws a line segment to connect the first lower-level curve to the next one.

The algorithm then calls itself again to draw the next subcurve. This time it keeps `dx` and `dy` in their original positions so that the second subcurve is not rotated.

The algorithm draws another connecting line segment and then calls itself, again keeping `dx` and `dy` in their original positions so that the third subcurve is not rotated.

The algorithm draws another connecting line segment and then calls itself a final time. This time it replaces `dx` with `-dy` and `dy` with `-dx` so that the smaller curve is rotated -90 degrees.

Sierpiński Curve

Like the Koch and Hilbert curves, the Sierpiński curve (named after Polish mathematician Waław Sierpiński, 1882–1969) draws a higher-level curve by using lower-level copies of itself. Unlike the other curves, however, the simplest method of drawing a Sierpiński curve uses four indirectly recursive routines that call each other rather than a single routine.

Figure 9.10 shows level 0, 1, 2, and 3 Sierpiński curves. In the level 1, 2, and 3 curves, the lines connecting the lower-level curves are gray so that you can see how the pieces are connected to build the higher-level curves.

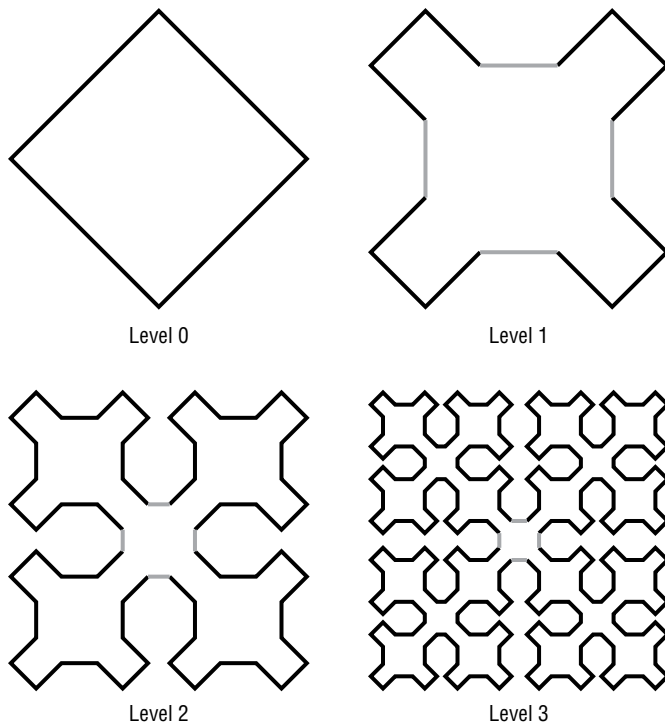


Figure 9.10: A Sierpiński curve is made up of four smaller Sierpiński curves.

Figure 9.11 shows the pieces of the level 1 Sierpiński curve. The curve consists of four sides that are drawn by four different routines and connected by line segments. The four routines draw curves that move the current drawing position in the right, down, left, and up directions. For example, the Right routine draws a series of segments that leaves the drawing position moved to the right. In Figure 9.11, the connecting line segments are drawn in gray.

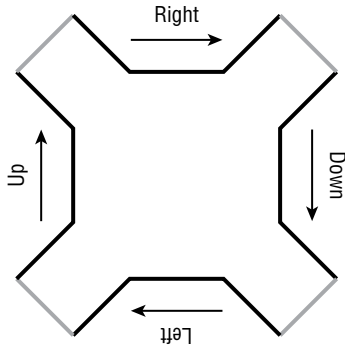


Figure 9.11: The level 1 Sierpiński curve is made up of pieces that go right, down, left, and up.

To draw a higher-level version of one of the pieces of the curve, the algorithm breaks that piece into smaller pieces that have a lower level. For example, Figure 9.12 shows how you can make a depth 2 Right piece out of four depth 1 pieces. If you study Figure 9.10, you can figure out how to make the other pieces.

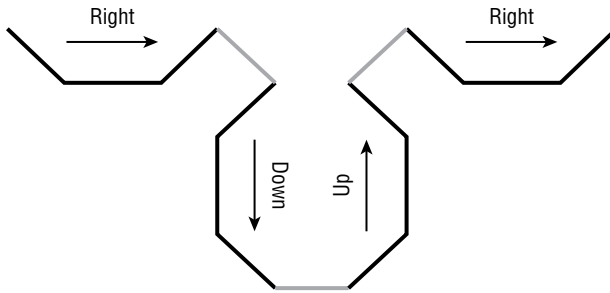


Figure 9.12: The level 2 Right piece is made up of pieces that go right, down, up, and right.

The following pseudocode shows the main algorithm:

```
// Draw a Sierpiński curve.
Sierpinski(Integer: depth, Float: dx, Float: dy)
    SierpRight(depth, dx, dy)
    DrawRelative(dx, dy)
    SierpDown(depth, dx, dy)
    DrawRelative(-dx, dy)
    SierpLeft(depth, dx, dy)
    DrawRelative(-dx, -dy)
    SierpUp(depth, dx, dy)
    DrawRelative(dx, -dy)
End Sierpiński
```

The algorithm calls the methods `SierpRight`, `SierpDown`, `SierpLeft`, and `SierpUp` to draw the pieces of the curve. Between those calls, the algorithm calls

`DrawRelative` to draw the line segments connecting the pieces of the curve. As was the case with the Hilbert curve, the `DrawRelative` method draws from that current position to a new point relative to that position and updates the current position. These calls to `DrawRelative` are the only places where the algorithm actually does any drawing.

The following pseudocode shows the `SierpRight` algorithm:

```
// Draw right across the top.
SierpRight(Integer: depth, Float: dx, Float: dy)
    If (depth > 0) Then
        depth = depth - 1

        SierpRight(depth, dx, dy)
        DrawRelative(dx, dy)
        SierpDown(depth, dx, dy)
        DrawRelative(2 * dx, 0)
        SierpUp(depth, dx, dy)
        DrawRelative(dx, -dy)
        SierpRight(depth, dx, dy)
    End If
End SierpRight
```

You can follow this method's progress in Figure 9.12. First this method calls `SierpRight` to draw a piece of the curve moving to the right at a smaller depth of recursion. It then draws a segment down and to the right to connect to the next piece of the curve.

Next the method calls `SierpDown` to draw a piece of the curve moving downward at a smaller depth of recursion. It draws a segment to the right to connect to the next piece of the curve.

The method then calls `SierpUp` to draw a piece of the curve moving upward at a smaller depth of recursion. It draws a segment up and to the right to connect to the next piece of the curve.

The method finishes by calling `SierpRight` to draw the final piece of the curve moving to the right at a smaller depth of recursion.

Figuring out the other methods that draw pieces of the Sierpiński curve is left as an exercise for you.

Because the Sierpiński methods call each other multiple times, they are multiply and indirectly recursive. Coming up with a nonrecursive solution from scratch would be difficult.

APPROXIMATE ROUTING

The Hilbert and Sierpiński curves are two examples of *space-filling curves*. Intuitively, a space-filling curve is one that, in the limit, approaches arbitrarily closely to every point in the area that it covers. For example, suppose that you draw a Hilbert curve in the

square $-1 \leq x, y \leq 1$. If you pick any point in that rectangle, then for a sufficiently high level of recursion, the Hilbert curve passes within any desired distance of that point. (One of my old math professors used to say, “Pick *any* distance. Could be one over infinity billion! The curve will pass that close to the point.)

Space-filling curves such as these provide a simple method of approximate routing. Suppose that you need to visit a group of stops in a city. If you draw a Hilbert or Sierpiński curve over the map of the city, then you can visit the stops in the order in which they are visited by the curve. (You don’t need to drive along the curve. You just use it to generate the ordering.)

The result probably won’t be optimal, but it probably will be reasonably good. You can use it as a starting point for the traveling salesperson problem (TSP) described in Chapter 17, “Complexity Theory.”

Gaskets

A *gasket* is another type of self-similar fractal. To draw a gasket, you start with a geometric shape such as a triangle or square. If the desired depth of recursion is 0, the method simply fills the shape. If the desired depth is greater than 0, the method subdivides the shape into smaller, similar shapes and then calls itself recursively to draw some but not all of them.

For example, Figure 9.13 shows depth 0 through 3 triangular gaskets that are often called Sierpiński gaskets (or Sierpiński sieves or Sierpiński triangles).

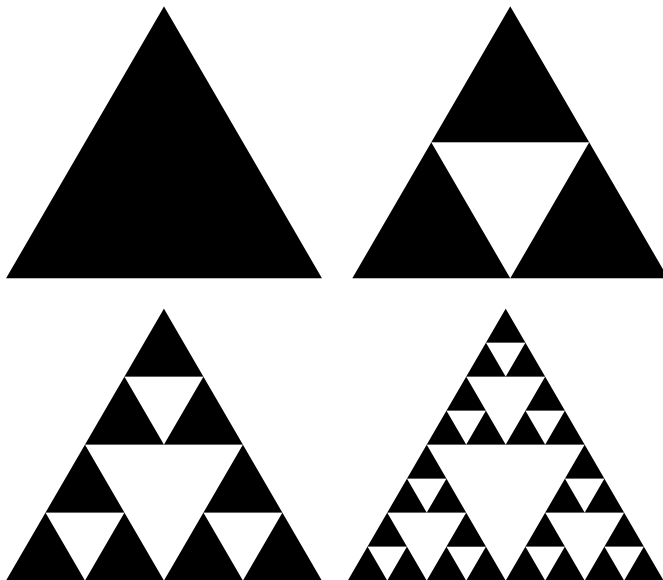


Figure 9.13: To create a Sierpiński gasket, divide a triangle into four pieces and recursively color the three at the triangle’s corners.

Figure 9.14 shows a square gasket that is often called the Sierpiński carpet. (Yes, named after the same Sierpiński whose name is attached to the Sierpiński curve. He studied many of these sorts of shapes, so his name is attached to several of them.)

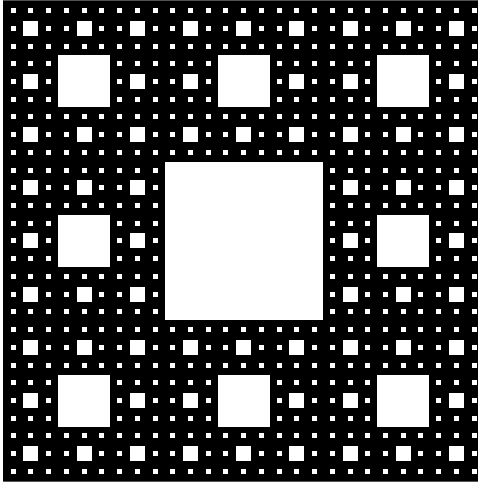


Figure 9.14: To create a Sierpiński carpet, divide a square into nine pieces, remove the center one, and recursively color the rest.

Writing low-level pseudocode to draw the Sierpiński gasket and carpet are left as Exercises 10 and 11 at the end of this chapter.

The Skyline Problem

In the *skyline problem*, you are given a collection of rectangles that have a common baseline, and you need to find an outline for them. Figure 9.15 shows a collection of rectangles on the left and their skyline on the right. The small circles show places where the skyline moves to a new height.

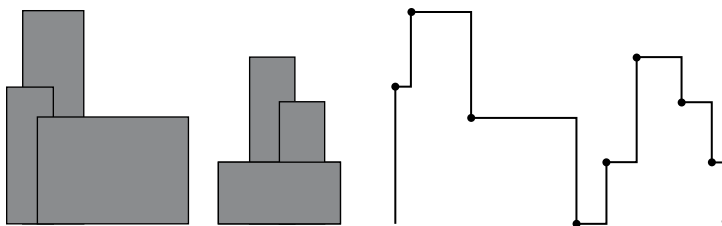


Figure 9.15: The skyline problem produces an outline similar to the outline of skyscrapers in a city.

The following two sections describe different approaches to the skyline problem.

Lists

One relatively straightforward approach to solving this problem is to create a `HeightChange` object to represent places where a rectangle begins or ends. You can then sort those objects by their X coordinates and loop through them in order from left to right.

Each time you reach a `HeightChange` object, you update the skyline's current height appropriately. For example, if the skyline's current height is lower than the height given by the new `HeightChange` object, you increase the skyline's current height.

Unfortunately, if the new height is lower than the skyline's current height, you have some extra work to do. In that case, you need to know which rectangles have been started but not yet finished. The skyline's new height will be the largest height of any of those "active" rectangles.

For example, consider the group of rectangles on the left in Figure 9.15. When the tallest rectangle ends, the new height of the skyline should be the height of the short, wide rectangle because that rectangle has been started but not yet finished.

You can keep track of the active rectangles in several ways. One of the simplest is to add each rectangle to an active list when you reach its starting `HeightChange` object. When you need to find the tallest active rectangle, you simply loop through the list. Later, when you reach an active rectangle's ending `HeightChange` object, you remove it from the list.

If there are N rectangles, then the algorithm needs $O(N)$ `HeightChange` objects to represent the places where the rectangles start and stop. Sorting those objects takes $O(N \log N)$ steps.

The algorithm then processes the $O(N)$ sorted objects. Each time an object's height is smaller than the skyline's current height, you need to search through the list of active objects. That list could hold up to $O(N)$ objects at a time, so that step might take up to $O(N)$ steps. Because the algorithm performs that $O(N)$ search for $O(N)$ objects, the total time for the processing step is $O(N^2)$.

That makes the total time to build the skyline $O(N) + O(\log N) + O(N^2) = O(N^2)$.

The following pseudocode shows the algorithm at a high level:

```
List<Point>: MakeSkyline(List<Rectangle>: rectangles)
    <Make HeightChange objects to represent each rectangle's
      start and stop.>
    <Place the HeightChange objects in a list named changes.>
    <Sort the changes list.>

    <Make an empty list activeTops to hold active rectangles.>
    Integer: currentY = <ground height>
    For Each change In changes
        // See if we are starting or stopping a building.
        If (change.Starting) Then
```

```

        // Starting a building.
        <Add new points to increase the skyline's height.>
    Else
        // Ending a building.
        <Remove this rectangle from the active list.>
        <Find the tallest active rectangle.>
        <Add new points to the skyline to move to that
            building's height.>
    End If
Next change

Return <The skyline points>
End MakeSkyline

```

NOTE This algorithm talks about height as you would normally think of it in Figure 9.15. In most programming languages, however, Y coordinates usually increase downward. That means when the algorithm says, “Find the tallest active rectangle,” you actually need to look for the rectangle with an upper edge having the smallest Y coordinate.

In practice, this algorithm is relatively simple and fast, but it still has $O(N^2)$ performance.

One of the most potentially time-consuming operations performed by the algorithm is finding the tallest currently active rectangle. Because the rectangles are stored in a simple list, that task takes $O(N)$ steps. If you store the active rectangles in a priority queue instead of a list, then you can find the tallest item in a single step, plus $O(\log N)$ steps to rearrange the queue. With that change, the algorithm’s run time becomes $O(N) + O(\log N) + O(N \log N) = O(N \log N)$.

Divide and Conquer

The algorithm described in the preceding section processes `HeightChange` objects in order from left to right. It’s reasonably straightforward, but it requires the algorithm to examine the rectangles that are active at any given step. Because there could be many active rectangles, that process can take a long time.

Another strategy is to use a divide-and-conquer approach, similar to the one used by mergesort. Instead of considering the rectangles’ edges in sorted order, you divide the rectangles into two groups, recursively create skylines for the two groups, and then merge those two skylines.

The hardest part of this approach is merging two skylines. For example, suppose you are working on the rectangles shown on the left in Figure 9.16. Also suppose that the algorithm has created the two skylines on the right, one for the lighter set of rectangles and one for the darker set of rectangles.

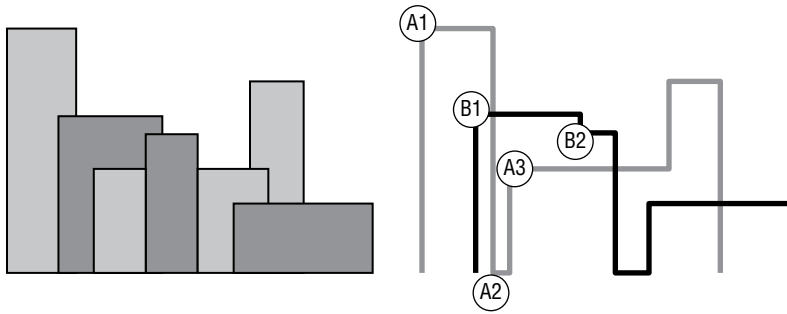


Figure 9.16: The light and dark skylines on the right correspond to the light and dark rectangles on the left.

To merge the two skylines, make three variables: one to keep track of the new merged skyline's height and two to keep track of the smaller skylines' left edges. Initially, the left edge variables indicate the leftmost points on the skylines, which are labeled A1 and B1 in the figure. (The labels in the figure show places where the skylines' heights change.)

The algorithm compares the X coordinates of the left edge variables and finds that point A1 has a smaller X coordinate. The result skyline currently has height 0, so the algorithm updates it to the height of point A1. It also moves the left edge variable for the lighter skyline to the next point on that skyline, which is point A2.

The algorithm again compares the left edge variables, which now refer to points A2 and B1. Point B1 has the smaller X coordinate, so the algorithm uses that one next. The height at point B1 is less than the height of the other skyline, so the algorithm does not change the result skyline's height.

This is one place where this algorithm has an advantage over the previous one. At the corresponding point in the previous algorithm, the program needed to search a list of active rectangles to see which one is currently highest. The new algorithm needs to consider only two partial skylines at a time.

After considering point B1, the algorithm moves the darker skyline's left edge variable to point B2. It then considers points A2 and B2. Point A2 comes first. That point's height is less than the height of the dark skyline, so the algorithm moves the result skyline's height to the height of the dark skyline.

The algorithm continues tracing its way through the two partial skylines until it has processed them both.

The following pseudocode shows the main `MakeSkyline` method, which implements the algorithm except for the merge step:

```
// Make the skyline points.
List<Point>: MakeSkyline(List<Rectangle>: rectangles,
    Integer: mini, Integer: maxi)
    // See if we need to process a single rectangle.
    If (mini == maxi) Then
        <Make and return a skyline representing the single rectangle.>
    End If
```



```

// Process the two halves.
Integer: midi = (mini + maxi) / 2
List<Point>: skyline1 = MakeSkyline(rectangles, mini, midi)
List<Point>: skyline2 = MakeSkyline(rectangles, midi + 1, maxi)

// Merge and return the result.
Return MergeSkylines(skyline1, skyline2)
End MakeSkyline

```

This method's `rectangles` parameter holds the rectangles that define the skyline. The `mini` and `maxi` parameters give the minimum and maximum indices of the rectangles that this call to `MakeSkyline` should consider.

If `mini` equals `maxi`, then the method is building a skyline for a single rectangle. In that case, it simply builds the one-rectangle skyline and returns it.

If `mini` does not equal `maxi`, then the method sets variable `midi` to the average of `mini` and `maxi`. It then divides the rectangles into two groups and recursively calls itself to make skylines for the two groups. When the recursive calls return, the method calls the `MergeSkylines` method to merge the two skylines and then returns the combined result.

The following pseudocode shows the `MergeSkylines` method:

```

// Merge two skylines.
List<Point>: MergeSkylines(List<Point>: skyline1,
List<Point>: skyline2)
    List<Point>: results = new List<Point>()

    <Set index1 and index2 to the indices of the leftmost
    skyline points.>
    <Set y1 and y2 equal to the ground's Y coordinate.>
    <Set currentY equal to the ground's Y coordinate.>
    While (index1 < skyline1.Length) Or (index2 < skyline2.Length)
        <Compare the next point in each skyline.>
        <Add the appropriate point to results.>
        <Update currentY, index1, and index2.>
    End While

    // Add any remaining points.
    For i = index1 To skyline1.Length - 1
        <Add skyline1[index1] to results.>
        index1++
    Next i
    For i = index2 To skyline2.Length - 1
        <Add skyline2[index2] to results.>
        index2++
    Next i

    Return results
End MergeSkylines

```

This method performs some initialization and then enters a loop that executes as long as there are unmerged points in both of the input skylines. Inside the loop, it compares the two skylines' left edge points and the `currentY` value to determine the result skyline's correct height. It then updates `index1`, `index2`, and `currentY` appropriately.

After the method finishes processing one of the input skylines, it adds the other skyline's remaining points to the result skyline. It then returns the result.

This algorithm's structure is similar to that of the mergesort algorithm, so it has a similar run time of $O(N \log N)$. In practice, both this algorithm and the previous one are quite fast, so the difference isn't obvious until you use a large number of rectangles. In one set of tests, both algorithms were able to build a skyline from 100,000 rectangles in less than one second.

With 500,000 rectangles, the difference was obvious. With that many rectangles, the divide and conquer algorithm took around 0.57 seconds while the active list algorithm took roughly 16.79 seconds.

Backtracking Algorithms

Backtracking algorithms use recursion to search for the best solution to complicated problems. These algorithms recursively build partial test solutions to solve the problem. When they find that a test solution cannot lead to a usable final solution, they backtrack, discarding that test solution and continuing the search from an earlier test solution.

Backtracking is useful when you can incrementally build partial solutions, and you can sometimes quickly determine that a partial solution cannot lead to a complete solution. In that case, you can stop improving that partial solution, backtrack to the previous partial solution, and continue the search from there.

The following pseudocode shows the general backtracking approach at a high level:

```
// Explore this test solution.
// Return false if it cannot be extended to a full solution.
// Return true if a recursive call to LeadsToSolution finds
// a full solution.
Boolean: LeadsToSolution(Solution: test_solution)
    // If we can already tell that this partial solution cannot
    // lead to a full solution, return false.
    If <test_solution cannot solve the problem> Then Return false

    // If this is a full solution, return true.
    If <test_solution is a full solution> Then Return true
```

```

// Extend the partial solution.
Loop <over all possible extensions to test_solution>
    <Extend test_solution>

    // Recursively see if this leads to a solution.
    If (LeadsToSolution(test_solution)) Then Return true

    // This extension did not lead to a solution. Undo the change.
    <Undo the extension>
End Loop

// If we get here, this partial solution cannot
// lead to a full solution.
Return false
End LeadsToSolution

```

The `LeadsToSolution` algorithm takes as a parameter whatever data it needs to keep track of a partial solution. It returns `true` if that partial solution leads to a full solution.

The algorithm begins by testing the partial solution to see if it is illegal. If the test solution so far cannot lead to a feasible solution, the algorithm returns `false`. The calling instance of `LeadsToSolution` abandons this test solution and works on others.

If the test solution looks valid so far, the algorithm loops over all the possible ways that it can extend the solution toward a full solution. For each extension, the algorithm calls itself recursively to see whether the extended solution will work. If the recursive call returns `false`, that extension doesn't work, so the algorithm undoes the extension and tries again with a new extension.

If the algorithm tries all possible extensions to the test solution and cannot find a feasible solution, it returns `false` so that the calling instance of `LeadsToSolution` can abandon this test solution.

You can think of the quest for a solution as a search through a hypothetical decision tree. Each branch in the tree corresponds to a particular decision attempting to solve the problem. For example, an optimal chess game tree would contain branches for every possible move at a given point in the game. If you can use a relatively quick test to realize that a partial solution cannot produce a full solution, then you can trim the corresponding branch off of the tree without searching it exhaustively. That can remove huge chunks from the tree and save a lot of time. (The idea of decision trees is described further in Chapter 12.)

The following sections describe two problems that have natural backtracking algorithms: the eight queens problem and the knight's tour problem. When you study the algorithms for those problems, the more general backtracking approach described here should be easier to understand.

Eight Queens Problem

In the eight queens problem, the goal is to position eight queens on a chessboard so that none of them can attack any of the other queens. In other words, no two queens can be in the same row, column, or diagonal. Figure 9.14 shows one solution to the eight queens problem.

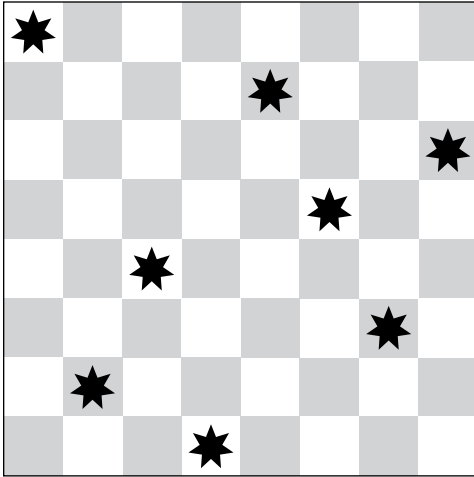


Figure 9.17: In the eight queens problem, you must position eight queens on a chessboard so that none can attack any of the others.

One way to solve this problem would be to try every possible arrangement of eight queens on a chessboard. Unfortunately, $\binom{64}{8} = 4,426,165,368$ arrangements are possible. You could enumerate all of them, but doing so would be time-consuming.

COUNTING COMBINATIONS

The reason why there are so many possible arrangements of queens is that you can position 8 queens in any of 64 squares. The queens are identical, so it doesn't matter which queen you use in a given location. That means the number of possible arrangements is the same as the number of ways that you can pick 8 squares out of the possible 64.

The number of selections of k items from a set of n without duplicates is given by the binomial coefficient. That value is written $\binom{n}{k}$ and is pronounced "n choose k." You can calculate the value with this formula:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

For example, this equation gives the number of different selections of three items from a set of five without duplicates:

$$\binom{5}{3} = \frac{5!}{3!(5-3)!} = \frac{5!}{3!2!} = \frac{120}{6 \times 2} = 10$$

The number of selections of k items from a set of n allowing duplicates (if you are allowed to pick the same item more than once in a selection) is given by this formula:

$$\binom{n+k-1}{k}$$

For example, this equation gives the number of different selections of three items from a set of five allowing duplicates:

$$\binom{5+3-1}{3} = \binom{7}{3} = \frac{7!}{3!(7-3)!} = \frac{7!}{3!4!} = \frac{5040}{6 \times 24} = 35$$

In the eight queens problem, you need to find the number of ways to pick 8 of the squares without duplicates (you can't put more than one queen on the same square). That means the number of possible selections is as follows:

$$\binom{64}{8} = \frac{64!}{8!(64-8)!} = \frac{64!}{8!56!} = 4,426,165,368$$

Backtracking works well for this problem because it allows you to eliminate certain possibilities from consideration. For example, you could start with a partial solution that places a queen on the board's upper-left corner. You might try adding another queen just to the right of the first queen, but you know that placing two queens on the same row isn't allowed. This means you can eliminate every possible solution that has the first two queens next to each other in the upper-left corner. The program can backtrack to the point before it added the second queen and search for more promising solutions.

This may seem like a trivial benefit. After all, you know a solution cannot have two queens side by side in the upper-left corner. However, $\binom{62}{6} = 61,474,519$ possible arrangements of eight queens on a chessboard have queens in those positions, so one backtracking step saves you the effort of examining more than 61 million possibilities.

In fact, if the first queen is placed in the upper-left corner, then no other queen can be placed in the same row, column, or diagonal. This means that there are a total of 21 places where the second queen cannot be placed. Eliminating all of those partial solutions removes almost 1.3 billion possible arrangements from consideration.

Later tests remove other partial solutions from consideration. For example, after you place the second queen somewhere legal, it further restricts where the third queen can be placed.

The following pseudocode shows how you can use backtracking to solve the eight queens problem:

```

Boolean: EightQueens(Boolean: spot_taken[,],
Integer: num_queens_positioned)
    // See If the test solution is already illegal.
    If (Not IsLegal(spot_taken)) Then Return false

    // See if we have positioned all of the queens.
    If (num_queens_positioned == 8) Then Return true

    // Extend the partial solution.
    // Try all positions for the next queen.
    For row = 0 to 7
        For col = 0 to 7
            // See if this spot is already taken.
            If (Not spot_taken[row, col]) Then
                // Put a queen here.
                spot_taken[row, col] = true

                // Recursively see if this leads to a solution.
                If (EightQueens(spot_taken, num_queens_positioned + 1))
                    Then Return true

                // The extension did not lead to a solution.
                // Undo the change.
                spot_taken[row, col] = false
            End If
        Next col
    Next row

    // If we get here, we could not find a valid solution.
    Return false
End EightQueens

```

The algorithm takes as a parameter a two-dimensional array of Booleans named `spot_taken`. The entry `spot_taken[row, col]` is true if there is a queen in row `row` and column `col`.

The algorithm's second parameter, `num_queens_positioned`, specifies how many queens have been placed in the test solution.

The algorithm starts by calling `IsLegal` to see if the test solution so far is legal. The `IsLegal` method, which isn't shown here, simply loops through the `spot_taken` array to see if there are two queens in the same row, column, or diagonal.

Next the algorithm compares `num_queens_positioned` with the total number of queens, 8. If all of the queens have been positioned, then this test solution is a full solution, so the algorithm returns true. (The `spot_taken` array is not

modified after that point, so when the first call to `EightQueens` returns, the array holds the solution.)

If this is not a full solution, the algorithm loops over all of the rows and columns. For each row/column pair, it checks `spot_taken` to see if that spot already contains a queen. If the spot doesn't hold a queen, the algorithm puts the next queen there and calls itself recursively to see if the extended solution leads to a full solution.

If the recursive call returns `true`, then it found a full solution, so this call also returns `true`.

If the recursive call returns `false`, the extended solution does not lead to a full solution, so the algorithm removes the queen from its new position and tries the next possible position.

If the algorithm tries all possible locations for the next queen and none of them work, this test solution (before the new queen is added) cannot lead to a full solution, so the algorithm returns `false`.

You can improve this algorithm's performance in a couple of interesting ways. See Exercises 13 and 14 at the end of this chapter for details.

Knight's Tour

In the *knight's tour problem*, the goal is to make a knight visit every position on a chessboard without visiting any square twice. A tour is considered *closed* if the final position is one move away from the starting position, and the knight could immediately start the tour again. A tour that is not closed is considered open.

NOTE In case you don't remember, a knight moves two squares horizontally or vertically and then one square perpendicularly from its current position, as shown in Figure 9.18.

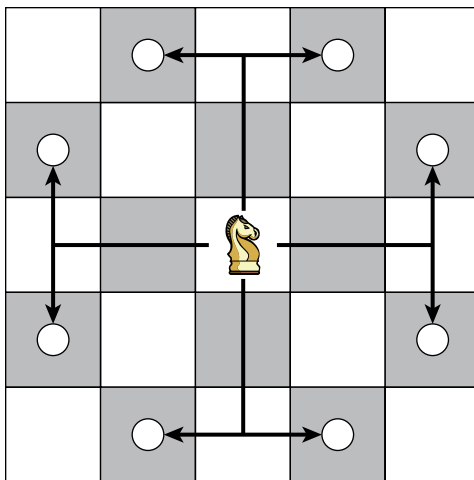


Figure 9.18: In chess, a knight can move to eight places if none of them lies off the board.

The following pseudocode shows a backtracking solution to the knight's tour problem:

```
// Move the knight to position [row, col]. Then recursively try
// to make other moves. Return true if we find a valid solution.
Boolean: KnightsTour(Integer: row, Integer: col,
Integer: move_number[,], Integer: num_moves_taken)
    // Move the knight to this position.
    num_moves_taken = num_moves_taken + 1
    move_number[row, col] = num_moves_taken

    // See if we have made all the required moves.
    If (num_moves_taken == 64) Then Return true

    // Build arrays to determine where legal moves are
    // with respect to this position.
    Integer: dRows[] = { -2, -2, -1, 1, 2, 2, 1, -1 }
    Integer: dCols[] = { -1, 1, 2, 2, 1, -1, -2, -2 }

    // Try all legal positions for the next move.
    For i = 0 To 7
        Integer: r = row + d_rows[i]
        Integer: c = col + d_cols[i]
        If ((r >= 0) And (r < NumRows) And
            (c >= 0) And (c < NumCols) And
            (move_number[r, c] == 0))
            Then
                // This move is legal and available. Make this move
                // and then recursively try other assignments.
                If (KnightsTour(r, c, move_number, num_moves_taken))
                    Then Return true
            End If
    Next i

    // This move didn't work out. Undo it.
    move_number[row, col] = 0

    // If we get here, we did not find a valid solution.
    return false
End KnightsTour
```

This algorithm takes as parameters the row and column where the knight should move next, an array named `move_number` that gives the number of the move when the knight visited each square, and the number of moves made so far.

The algorithm starts by recording the knight's move to the current square and incrementing the number of moves made. If the number of moves made is 64, then the knight has finished a tour of the board, so the algorithm returns `true` to indicate success.

If the tour is not finished, the algorithm initializes two arrays to represent the moves that are possible from the current square. For example, the first entries in the arrays are `-2` and `-1`, indicating that the knight can move from square `(row, col)` to `(row - 2, col - 1)` if that square is on the board.

Next the algorithm loops through all of the possible moves from the position `(row, col)`. If a move is on the board and has not already been visited in the test tour, the algorithm recursively calls itself to see if that move leads to a full solution.

If none of the possible moves from the current position leads to a solution, the algorithm sets `move_number[row, col] = 0` to undo the current move and returns `false` to indicate that moving the knight to square `(row, col)` does not lead to a solution.

Unfortunately, the knight's tour problem isn't as easy to constrain as the eight queens problem. With the eight queens problem, it's fairly easy to tell whether a new position is under attack by another queen and therefore unavailable for a new queen. In that case, you can eliminate that position from consideration. A queen also attacks many squares, so it restricts later queens significantly.

In the knight's tour, any position the knight can reach that has not yet been visited gives a new test solution. There may be some cases where you can easily conclude that a test solution won't work, such as if the board has an unvisited square that is more than one move away from any other unvisited square but recognizing that situation is difficult.

The fact that it's hard to eliminate test solutions early on means that the algorithm often follows a test solution for a long while before discovering that the solution is infeasible. A knight can have up to eight legal moves, so an upper bound on the number of potential tours is 8^{64} , or roughly 6.3×10^{57} . You can study the positions on the board to get a better estimate of the potential number of tours (for example, a knight in a corner has only two possible moves), but the total number of potential tours is an enormous number in any case.

All this means is that it's difficult to solve the knight's tour problem for a normal 8×8 chessboard. In one set of tests, a program solved the problem on a 6×6 board almost instantly and on a 7×6 board in about 2 seconds. It still hadn't found a solution on a 7×7 board after an hour.

Although solving the knight's tour using only backtracking is difficult, a particular heuristic solves the problem extremely well. A *heuristic* is an algorithm that often produces a good result but that is not guaranteed to produce the best possible result. For example, a driving heuristic might be to add 10 percent to expected travel time to allow for traffic delays. Doing so doesn't guarantee that you'll always be on time, but it increases the chances.

Way back in 1823, H. C. von Warnsdorff suggested an amazingly effective knight's tour heuristic. At each step, the algorithm should select the next possible move that has the lowest number of possible moves leading out of it.

For example, suppose that the knight is in a position where it has only two possible moves. If the knight makes the first move, then it will have five possible locations for its next move. If the knight makes the second move, then it will have only one possible move for its next move. In that case, the heuristic says the algorithm should try the second move first.

This heuristic is so good that it finds a complete tour with no backtracking for boards of size up to 75×75 . (In my tests, the program found a solution on a 57×57 board almost instantly and then crashed with a stack overflow on a 58×58 board.)

Selections and Permutations

A *selection* or *combination* is a subset of a set of objects. For example, in the set $\{A, B, C\}$, the subset $\{A, B\}$ is a selection. All the two-item selections for the set $\{A, B, C\}$ are $\{A, B\}$, $\{A, C\}$, and $\{B, C\}$.

In a selection, the ordering of the items doesn't matter, so $\{A, B\}$ is considered the same as $\{B, A\}$. You can think of this as being similar to a menu selection at a restaurant. Selecting a cheese sandwich and milk is the same as selecting milk and a cheese sandwich.

In contrast, a *permutation* is an ordered arrangement of a subset of items taken from a set. This is similar to a selection, except that the ordering matters. For example, (A, B) and (B, A) are two permutations of two items taken from the set $\{A, B, C\}$. All of the permutations of two items taken from the set $\{A, B, C\}$ are (A, B) , (A, C) , (B, A) , (B, C) , (C, A) , and (C, B) . (Notice the notation uses brackets $\{ \}$ for unordered selections and parentheses $()$ for ordered permutations.)

One other factor determines which groups of items are included in a particular kind of selection or permutation, that is, whether duplicates are allowed or not. For example, the two-item selections for the set $\{A, B, C\}$ allowing duplicates include $\{A, A\}$, $\{B, B\}$, and $\{C, C\}$ in addition to the other selections listed earlier.

The special case of a permutation that takes all of the items in the set and doesn't allow duplicates gives all of the possible arrangements of the items. For the set $\{A, B, C\}$, all the arrangements are (A, B, C) , (A, C, B) , (B, A, C) , (B, C, A) , (C, A, B) , and (C, B, A) . Many people think of the permutations of a set to be this collection of arrangements, rather than the more general case in which you may not be taking all of the items from the set and you may allow duplicates.

The following sections describe algorithms that you can use to generate selections and permutations with and without duplicates.

Selections with Loops

If you know how many items you want to select from a set when you are writing a program, you can use a series of `For` loops to generate combinations easily. For example, the following pseudocode generates all of the selections of three items taken from a set of five items allowing duplicates:

```
// Generate selections of 3 items allowing duplicates.
List<string>: Select3WithDuplicates(List<string> items)
    List<string>: results = New List<string>
    For i = 0 To <Maximum index in items>
        For j = i To <Maximum index in items>
            For k = j To <Maximum index in items>
                results.Add(items[i] + items[j] + items[k])
            Next k
        Next j
    Next i
    Return results
End Select3WithDuplicates
```

This algorithm takes as a parameter a list of `Strings`. It then uses three `For` loops to select the three choices that make up each selection.

Each loop starts with the current value of the previous loop's counter. For example, the second loop starts with `j` equal to `i`. That means the second letter chosen for the selection will not be a letter that comes before the first letter in the items. For example, if the items include the letters *A*, *B*, *C*, *D*, and *E*, and the first letter in the selection is *C*, then the second letter won't be *A* or *B*. That keeps the letters in the selection in alphabetical order and prevents the algorithm from selecting both $\{A, B, C\}$ and $\{A, C, B\}$, which are the same set.

Within the innermost loop, the algorithm combines the items selected by each loop variable to produce an output that includes all three selections.

Modifying this algorithm to prevent duplicates is simple. Just start each loop at the value 1 greater than the current value in the next outer loop. The following pseudocode shows the algorithm with this modification highlighted:

```
// Generate selections of 3 items without allowing duplicates.
List<string>: Select3WithoutDuplicates(List<string> items)
    List<string>: results = new List<string>()
    For i = 0 To <Maximum index in items>
        For j = i + 1 To <Maximum index in items>
            For k = j + 1 To <Maximum index in items>
                results.Add(items[i] + items[j] + items[k])
            Next k
        Next j
    Next i
```

```
        Next j
    Next i
    Return results
End Select3WithoutDuplicates
```

This time, each loop starts at 1 greater than the previous loop's current value, so a loop cannot select the same item as a previous loop and that prevents duplicates.

Selections with Duplicates

The problem with the algorithms described in the preceding section is that they require you to know how many items you will select when you write the code, and sometimes that may not be the case. If you don't know how many items are in the original set of items, the program can figure that out. However, if you don't know how many items to select, you can't program the right number of For loops.

You can solve this problem recursively. Each call to the algorithm is responsible for adding a single selection to the result. Then, if the result doesn't include enough selections, the algorithm calls itself recursively to make more. When the selection is complete, the algorithm does something with it, such as printing the list of items selected.

The following pseudocode shows a recursive algorithm that generates selections allowing duplicates:

```
// Generate combinations allowing duplicates.
SelectKofNwithDuplicates(Integer: index, Integer: selections[],
    Data: items[], List<List<Data>> results)
    // See if we have made the last assignment.
    If (index == <Length of selections>) Then
        // Add the result to the result list.
        List<Data> result = New List<Data>()
        For i = 0 To <Largest index in selections>
            result.Add(items[selections[i]])
        Next i
        results.Add(result)
    Else
        // Get the smallest value we can use for the next selection.
        Integer: start = 0 // Use this value the first time through.
        If (index > 0) Then start = selections[index - 1]

        // Make the next assignment.
        For i = start To <Largest index in items>
            // Add item i to the selection.
            selections[index] = i
```

```

        // Recursively make the other selections.
        SelectKofNwithDuplicates(index + 1, selections,
                                items, results)
    Next i
End If
End SelectKofNwithDuplicates

```

This algorithm takes the following parameters:

- `index` gives the index of the item in the selection that this recursive call should set. If `index` is 2, then this call to the algorithm fills in `selections [2]`.
- `selections` is an array to hold the indices of the selected items. For example, if `selections` holds two entries with values 2 and 3, then the selection includes the items with indices 2 and 3.
- `items` is an array of the items from which selections should be made.
- `results` is a list of lists of items representing the complete selections. For example, if a selection is {A, B, D}, then `results` holds a list including the indices of A, B, and D.

When the algorithm starts, it checks the index of the item in the selection that it should make. If this value is greater than the length of the `selections` array, the selection is complete, so the algorithm adds it to the `results` list.

If the selection is incomplete, the algorithm determines the smallest index in the `items` array that it could use for the next choice in the selection. If this call to the algorithm is filling in the first position in the `selections` array, it could use any value in the `items` array, so `start` is set to 0. If this call does not set the first item in the selection, the algorithm sets `start` to the index of the last value chosen.

For example, suppose the items are {A, B, C, D, E} and the algorithm has been called to fill in the third choice. Suppose also that the first two selected items were those with indices 0 and 2, so the current selection is {A, C}. In that case, the algorithm sets `start` to 3, so the items it considers for the next position have indices of 3 or greater. That makes it pick between D and E for this selection.

Setting `start` in this way keeps the items in the selection in order. In this example, that means the letters in the selection are always in alphabetical order. That prevents the algorithm from picking two selections such as {A, C, D} and {A, D, C}, which are the same items in different orders.

Having set `start`, the algorithm loops from `start` to the last index in the `items` array. For each of those values, the algorithm places the value in the `selections` array to add the corresponding item to the selection and then calls itself recursively to assign values to the other entries in the `selections` array.

Selections Without Duplicates

To produce selections without duplicates, you need to make only one minor change to the previous algorithm. Instead of setting the `start` variable equal to the index that was last added to the `selections` array, you set it to 1 greater than that index. That prevents the algorithm from selecting the same value again.

The following pseudocode shows the new algorithm with the modified line highlighted:

```
// Generate combinations allowing duplicates.
SelectKofNwithoutDuplicates(Integer: index, Integer: selections[],
Data: items[], List<List<Data>> results)
    // See if we have made the last assignment.
    If (index == <Length of selections>) Then
        // Add the result to the result list.
        List<Data> result = New List<Data>()
        For i = 0 To <Largest index in selections>
            Result.Add(items[selections[i]])
        Next i
        results.Add(result)
    Else
        // Get the smallest value we can use for the next selection.
        Integer: start = 0 // Use this value the first time through.
        If (index > 0) Then start = selections[index - 1] + 1

        // Make the next assignment.
        For i = start To <Largest index in items>
            // Add item i to the selection.
            selections[index] = i

            // Recursively make the other selections.
            SelectKofNwithoutDuplicates(
                index + 1, selections, items, results)
        Next i
    End If
End SelectKofNwithoutDuplicates
```

The algorithm works the same way as before, but this time each choice for an item in the selection must come after the one before it in the `items` list. For example, suppose the items are {A, B, C, D}, the algorithm has already chosen {A, B} for the partial selection, and now the algorithm has been called to make the third selection. In that case, the algorithm considers only the items that come after B, which are C and D.

Permutations with Duplicates

The algorithm for generating permutations is similar to the previous ones for generating selections. The following pseudocode shows the algorithm for generating permutations allowing duplicates:

```
// Generate permutations allowing duplicates.
PermuteKofNwithDuplicates(Integer: index, Integer: selections[],
Data: items[], List<List<Data>> results)
    // See if we have made the last assignment.
    If (index == <Length of selections>) Then
        // Add the result to the result list.
        List<Data> result = New List<Data>()
        For i = 0 To <Largest index in selections>
            Result.Add(items[selections[i]])
        Next i
        results.Add(result)
    Else
        // Make the next assignment.
        For i = 0 To <Largest index in items>
            // Add item i to the selection.
            selections[index] = i

            // Recursively make the other assignments.
            PermuteKofNwithDuplicates(index + 1, selections, items, results)
        Next i
    End If
End PermuteKofNwithDuplicates
```

The main difference between this algorithm and the earlier one for generating selections with duplicates is that this algorithm loops through all of the items when it makes its assignment instead of starting the loop at a start value. This allows the algorithm to pick items in any order, so it generates all of the permutations.

COUNTING PERMUTATIONS WITH DUPLICATES

Suppose you're making permutations of k out of n items allowing duplicates. For each item in a permutation, the algorithm could pick any of the n choices. It makes k independent choices (in other words, one choice does not depend on the previous choices), so there are $n * n * \dots * n = n^k$ possible permutations.

In the special case where you want to generate permutations that select all n of the n items, n^n results are possible.

Just as you can define selections without duplicates, you can define permutations without duplicates.

Permutations Without Duplicates

To produce permutations without duplicates, you need to make only one minor change to the preceding algorithm. Instead of allowing all of the items to be selected for each assignment, the algorithm excludes any items that have already been used.

The following pseudocode shows the new algorithm with the modified lines highlighted:

```
// Generate permutations not allowing duplicates.
PermuteKofNwithoutDuplicates(Integer: index, Integer: selections[],
    Data: items[], List<List<Data>> results)
    // See if we have made the last assignment.
    If (index == <Length of selections>) Then
        // Add the result to the result list.
        List<Data> result = New List<Data>()
        For i = 0 To <Largest index in selections>
            Result.Add(items[selections[i]])
        Next i
        results.Add(result)
    Else
        // Make the next assignment.
        For i = 0 To <Largest index in items>
            // Make sure item i hasn't been used yet.
            Boolean: used = false
            For j = 0 To index - 1
                If (selections[j] == i) Then used = true
            Next j

            If (Not used) Then
                // Add item i to the selection.
                selections[index] = i

                // Recursively make the other assignments.
                PermuteKofNwithoutDuplicates(
                    index + 1, selections, items, results)
            End If
        Next i
    End If
End PermuteKofNwithoutDuplicates
```

The only change is that this version of the algorithm checks that an item has not already been used in the permutation before adding it.

COUNTING PERMUTATIONS WITHOUT DUPLICATES

Suppose that you're making permutations of k out of n items without duplicates. For the first item in a permutation, the algorithm could pick any of the n choices. For the second item, it could pick any of the remaining $n - 1$ items. Multiplying the number of choices at each step gives the total number of possible permutations: $n \times (n - 1) \times (n - 2) \times \dots \times (n - k + 1)$.

In the special case where $k = n$, so that you are generating permutations that select all n of the items without duplicates, this formula becomes $n \times (n - 1) \times (n - 2) \times \dots \times 1 = n!$. This is the number most people think of as the number of permutations of a set.

Round-Robin Scheduling

Round-robin scheduling is a special kind of permutation. For a two-team (or two-player) sport, a *round-robin tournament* is a tournament where every team plays every other team once. A *round-robin schedule* consists of a collection of rounds during which every team plays another team. If the number of teams is odd, then one team in each round receives a *bye* when they don't play.

The result is a set of very specific permutations of the teams. To make the schedule as short as possible, no two teams should play against each other twice, and no team should have more than one bye. Each team either plays a new team or has a bye in each round. If the number of teams N is even, then there are no byes, so it takes $N - 1$ rounds for every team to play each of the other $N - 1$ teams. If N is odd, then it takes N rounds for every team to play each of the other $N - 1$ teams and have a bye.

For example, the following list shows a schedule for five teams:

Round 1

- Team 1 versus Team 4
- Team 2 versus Team 3
- Team 5 has a bye

Round 2

- Team 5 versus Team 3
- Team 1 versus Team 2
- Team 4 has a bye

Round 3

- Team 4 versus Team 2
- Team 5 versus Team 1
- Team 3 has a bye

Round 4

- Team 3 versus Team 1
- Team 4 versus Team 5
- Team 2 has a bye

Round 5

- Team 2 versus Team 5
- Team 3 versus Team 4
- Team 1 has a bye

One way that you could schedule a round robin tournament would be to try all possible random assignments until you find one that works. As you can probably imagine, that would be slow.

The following sections describe a better approach called the *polygon method*. The method works differently depending on whether you're scheduling an odd or even number of teams, so those cases are described separately.

Odd Number of Teams

To use the polygon method with an odd number of teams, draw a regular polygon with one vertex for each team and place the teams on the vertices. Then draw horizontal lines connecting teams that have the same Y coordinate. For example, Figure 9.19 shows the polygon for five teams labeled A, B, C, D, and E.

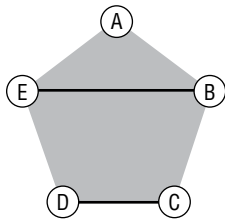


Figure 9.19: In the polygon method, horizontal lines represent matches in a tournament round.

The horizontal lines define the matches for the tournament's first round. In Figure 9.19, the lines indicate that team E plays team B, team D plays team C, and team A at the top of the polygon gets a bye in the round.

Now rotate the lines one position around the polygon's center. The lines now show the matches for round 2. Continue rotating the lines until they have defined all the tournament's matches.

More precisely, if there are N teams, then the polygon has N sides, the schedule includes N rounds, you rotate the lines $N - 1$ times, and you rotate the lines by $360 / N$ degrees between rounds.

Figure 9.20 shows the rotations for five teams. The lines are rotated $5 - 1 = 4$ times by $360 / 5 = 72^\circ$ each time.

The rotations shown in Figure 9.20 produce the following tournament schedule:

Round 1

- B versus E
- C versus D
- A has a bye

Round 2

- A versus C
- D versus E
- B has a bye

Round 3

- A versus E
- B versus D
- C has a bye

Round 4

- A versus B
- C versus E
- D has a bye

Round 5

- A versus D
- B versus C
- E has a bye

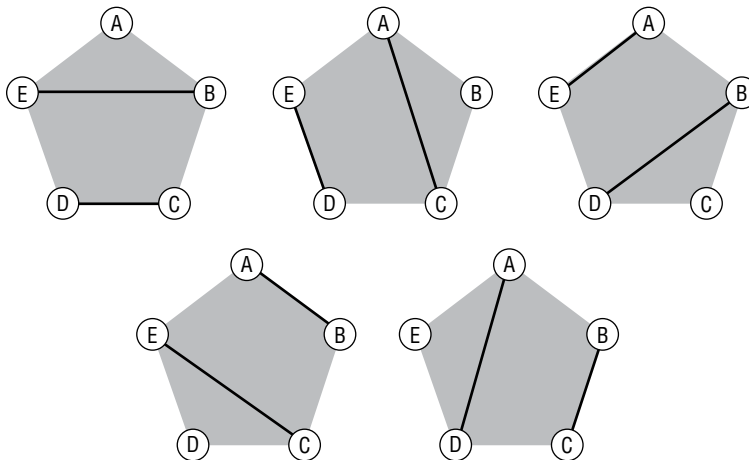


Figure 9.20: For N teams, you rotate the lines $360 / N$ degrees between rounds.

This technique clearly produces some sort of schedule. Now you need to ask, “Is this schedule valid and optimal?” To be valid and optimal, the schedule needs to satisfy the following requirements:

- In each round, each team plays exactly one other team or has a bye.
- Every team has exactly one bye.
- No team plays the same opponent twice.

The way the lines are constructed guarantees that the first requirement is satisfied. The number of teams is odd, so there is always one team that is not connected to a line, and that team gets the bye. Every other team is connected to exactly one line, so it plays exactly one opponent.

The second requirement is satisfied because, each time you rotate the lines, a different team is *not* connected to a line and therefore gets a bye. Because there are N arrangements, each team gets a turn being the one that gets the bye.

Another way to think of this is to leave the lines stationary and rotate the teams instead. (Whether you rotate the vertices or lines is just a matter of perspective. The resulting matchups are the same.) In that case, each team sits at the top of the polygon during one round, and that's when it gets its bye. Either way you think about it, this shows that every team gets exactly one bye, so the second requirement is satisfied.

The third requirement is a bit trickier. It is satisfied because each of the rotations produces different lines with no repetitions or reflections. For example, the first pentagon shown in Figure 9.20 includes a line between teams B and E, and no later rotation includes a line between those teams.

We know that there are no repeated lines because the lines are rotated by different angles after each round. After K rounds, the lines are rotated by $360 \times K / N$ degrees. Here $K < N$, so this angle is always less than 360° , and we know that the lines have not rotated all the way around to their original angles.

There's one other case to consider: the rotation angle might be 180° . In that case, the lines would be mirror images of their original positions. For example, the first pentagon in Figure 9.20 includes a line from team E to team B. If we rotate the lines by 180° , then the result would contain a line from team B to team E and that would represent a repeated match.

Fortunately, that cannot happen because N is odd. After K rounds, the lines are rotated by the angle $360 \times K / N$ degrees. If that angle is 180° , then $360 \times K / N = 180$. Solving this equation for N gives $N = 2K$. Because K is an integer, that means N is even, which contradicts our assumption that N is odd. That means there is no value for K that will rotate the lines by 180° , at least as long as N is odd. The following section explains how to schedule a tournament when N is even.

Even Number of Teams

Scheduling a tournament for an even number of teams is easy if you know how to handle an odd number of teams. Simply remove one of the teams and schedule the remaining teams as described in the preceding section. Then replace the byes with the team you removed.

For example, suppose you want to schedule a tournament for six teams labeled A through F. First build a schedule for the five teams A through E. Then replace the byes with matches against team F.

Implementation

The previous sections show that the polygon method works. Fortunately, you don't actually need to draw a bunch of polygons to implement the method. Instead, think of the teams as sitting in an array that wraps around halfway through, as shown in Figure 9.21.

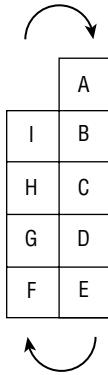


Figure 9.21: You can use a wrapped array to model the polygon method.

You can see how the elements in the array map to the polygon's vertices. The rows in Figure 9.21 correspond to the horizontal rows of vertices in Figure 9.20, so they give you the team matchups. For example, the first row in Figure 9.21 (not counting the odd team A at the top, which gets the bye) tells you that team I plays team B.

When you're ready for the next round, rotate the teams in the array, wrapping the last team back into the first position. Figure 9.22 shows the array during the second round.

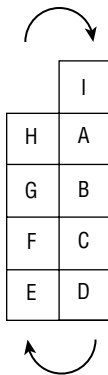


Figure 9.22: After each round, move the teams down one position in the array, wrapping the last team around to the beginning.

Understanding the polygon method takes some work, but the algorithm is surprisingly short. The following pseudocode shows the algorithm that schedules an odd number of teams. The `MatchUp` class simply holds two teams that should play each other. (In C# Python you could use a tuple instead of a class.)

```
// Find a round robin schedule for the teams.
List Of List Of MatchUp: ScheduleRoundRobinOdd(
    List Of String: teams, String: bye_team)
    Integer: num_teams = <length of teams>
    Integer: mid = num_teams / 2

    // Loop.
    List Of List Of MatchUp: schedule = New List Of List Of MatchUp()
    For i = 0 to num_teams - 1
        // Save this arrangement.
        List Of MatchUp: round = New List Of MatchUp()
        For j = 1 to mid
            round.Add(New MatchUp(teams[j], teams[num_teams - j]))
        Next j
        round.Add(New MatchUp(teams[0], bye_team))
        schedule.Add(round)

        // Rotate.
        teams.Insert(0, teams[num_teams - 1])
        teams.RemoveAt(num_teams)
    Next i

    return schedule
End ScheduleRoundRobinOdd
```

The algorithm calculates the number of teams and the index of the array's halfway point. The code assumes that integer division rounds down, so the midpoint index `mid` is the position of the last team in the first column of the wrapped array. For example, in Figure 9.21, `mid` is that index of team E.

The algorithm then creates a list to hold the tournament's rounds and enters a loop to generate the rounds.

Inside the loop, the algorithm creates a list to hold the matches for the current round. It then loops through the rows in the array, as shown in Figure 9.21, and saves the pairs of teams in the `round` list. This is the trickiest part of the algorithm and is basically bookkeeping.

After it finishes processing the round's matchups, the algorithm adds another matchup for the odd team (team A in Figure 9.21). It matches that team with the team in the `bye_team` parameter. That value should be either a team or a special value (such as `null`, `None`, or the string `BYE`) representing a bye. (You'll see how that works shortly.)

After it has built the round, the algorithm adds it to the `schedule` list.

Next, the algorithm rotates the array. To do that, it adds a copy of the last team to the beginning of the `teams` list and then removes the last team from the list. This operation is easy if the teams are stored in a queue. Then you can simply dequeue the first item and enqueue it at the end of the queue.

If the teams are stored in an array, then you'll need to move each of the teams into its new position.

The following pseudocode shows how you can use the `ScheduleRoundRobinOdd` method to build a schedule for any number of teams:

```
// Find a round robin schedule for the teams.
List Of List Of MatchUp: ScheduleRoundRobin(List Of String: teamList)
    // Copy the team list.
    List Of String: teams = <copy of teamList>

    // See if the number of teams is odd or even.
    String: byeTeam = "BYE"
    If (teams.Count % 2 == 0) Then
        // Even. Use the first item as the bye team.
        byeTeam = teams[0]
        teams.RemoveAt(0)
    End If

    // Make the schedule.
    return ScheduleRoundRobinOdd(teams, byeTeam)
End ScheduleRoundRobin
```

This algorithm makes a copy of the team list and works on the copy. It then determines whether the list contains an even or odd number of teams. If the number of teams is odd, it sets the variable `bye_team` to the string `BYE`. If the number of teams is even, it sets the variable `bye_team` to the first team and removes that team from the list.

The algorithm then calls `ScheduleRoundRobinOdd` to build a schedule for the team list. It passes that algorithm the `bye_team` variable so that it can pair unmatched teams with either the original first team or the special team named `BYE`.

The `ScheduleRoundRobin` algorithm then simply returns the schedule returned by `ScheduleRoundRobinOdd`.

Recursion Removal

Recursion makes some problems easier to understand. For example, the recursive solution to the Tower of Hanoi problem is simple and elegant.

Unfortunately, recursion has some drawbacks. Sometimes it leads to solutions that are natural but inefficient. For example, the recursive algorithm for

generating Fibonacci numbers requires that the program calculate the same values many times. This slows it down so much that calculating more than the 50th or so value is impractical.

Other recursive algorithms cause a deep series of method calls, which can exhaust the call stack. The knight's tour with Warnsdorff's heuristic demonstrates that problem. The heuristic can solve the knight's tour problem for boards up to 57×57 on my computer, but beyond that it exhausts the call stack, at least in C#.

Fortunately, you can do a few things to address these problems. The following sections describe some approaches that you can take to restructure or remove recursion to improve performance.

Tail Recursion Removal

Tail recursion occurs when the last thing a singly recursive algorithm does before returning is call itself. For example, consider the following implementation of the Factorial algorithm:

```
Integer: Factorial(Integer: n)
    If (n == 0) Then Return 1

    Integer: result = n * Factorial(n - 1)
    Return result
End Factorial
```

The algorithm starts by checking to see whether it needs to call itself recursively or whether it can simply return the value 1. If the algorithm must call itself, it does so, multiplies the returned result by n , and returns the result.

You can convert this recursive version of the algorithm into a nonrecursive version by using a loop. Within the loop, the algorithm performs whatever tasks the original algorithm did.

Before the end of the loop, the algorithm should set its parameters to the values they had during the recursive call. If the algorithm returns a value, as the Factorial algorithm does, you need to create a variable to keep track of the return value.

When the loop repeats, the parameters are set for the recursive call, so the algorithm does whatever the recursive call did.

The loop should end when the condition occurs that originally ended the recursion.

For the Factorial algorithm, the stopping condition is $n == 0$ so that condition controls the loop.

When the algorithm calls itself recursively, it decreases its parameter n by 1, so the non-recursive version should also decrease n by 1 before the end of the loop.

The following pseudocode shows the new nonrecursive version of the Factorial algorithm:

```
Integer: Factorial(Integer: n)
    // Make a variable to keep track of the returned value.
    // Initialize it to 1 so we can multiply it by returned results.
    // (The result is 1 if we do not enter the loop at all.)
    Integer: result = 1

    // Start a loop controlled by the recursion stopping condition.
    While (n != 0)
        // Save the result from this "recursive" call.
        result = result * n

        // Prepare for "recursion."
        n = n - 1
    Loop

    // Return the accumulated result.
    Return result
End Factorial
```

This algorithm looks a lot longer than it really is because of all the comments.

Removing tail recursion is straightforward enough that some compilers can do it automatically to reduce stack space requirements.

Of course, the problem with the Factorial algorithm isn't the depth of recursion, it's the fact that the results become too big to store in data types of fixed size. Tail recursion is still useful for other algorithms and usually improves performance, because checking a `While` loop's exit condition is faster than performing a recursive method call.

Dynamic Programming

Unfortunately, tail recursion won't help the Fibonacci algorithm for two reasons. First, it is multiply recursive, so tail recursion doesn't really apply. Perhaps, more importantly, its problem isn't great depth of recursion. Its problem is that it calculates too many intermediate results repeatedly, so it takes a long time to calculate results.

One solution to this problem is to record values as they are calculated so that the algorithm doesn't need to calculate them again later. The following pseudocode shows one way to do that:

```
// Calculated values.
Integer: FibonacciValues[100]

// The maximum value calculated so far.
Integer: MaxN
```

```

// Set Fibonacci[0] and Fibonacci[1].
InitializeFibonacci()
    FibonacciValues[0] = 0
    FibonacciValues[1] = 1
    MaxN = 1
End InitializeFibonacci

// Return the nth Fibonacci number.
Integer: Fibonacci(Integer: n)
    // If we have not yet calculated this value, calculate it.
    If (MaxN < n) Then
        FibonacciValues[n] = Fibonacci(n - 1) + Fibonacci(n - 2)
        MaxN = n
    End If

    // Return the calculated value.
    Return FibonacciValues[n]
End Fibonacci

```

This algorithm starts by declaring a globally visible `FibonacciValues` array to hold calculated values. The variable `MaxN` keeps track of the largest value `N` for which `Fibonacci(N)` has been stored in the array.

Next, the algorithm defines an initialization method called `InitializeFibonacci`. The program must call this method to set the first two Fibonacci values before it calls the `Fibonacci` function.

The `Fibonacci` function compares `MaxN` to its input parameter `n`. If the program has not yet calculated the `n`th Fibonacci number, it recursively calls itself to calculate that value, stores it in the `FibonacciValues` array, and updates `MaxN`.

Next the algorithm simply returns the value stored in the `FibonacciValues` array. At this point, the algorithm knows the value is in the array either because it was before or because the previous lines of code put it there.

In this program, each Fibonacci number is calculated only once. After that, the algorithm simply looks it up in the array instead of recursively calculating it.

NOTE This approach is called *dynamic programming*. The technique probably seemed dynamic when it was invented in the 1950s, but it seems downright static compared to modern techniques such as neural networks, genetic programming, and other machine learning techniques that basically reprogram themselves on the fly.

This approach solves the original Fibonacci algorithm's problem by letting it avoid calculating intermediate values a huge number of times. The original algorithm can calculate `Fibonacci(44)` in about a minute on my computer and cannot reasonably calculate values that are much larger. The new algorithm can calculate `Fibonacci(92)` almost instantly. It cannot calculate `Fibonacci(93)` in C# because the result doesn't fit in a 64-bit-long integer. If you use the `BigInteger` data type or integers in Python, the algorithm can easily calculate `Fibonacci(100)` or more.

Bottom-Up Programming

Dynamic programming makes the Fibonacci algorithm much faster, but it doesn't remove the recursion. It allows the program to calculate much larger values, but that also means it can enter deeper levels of recursion and exhaust the stack space.

To solve that problem, you need to remove the recursion. You can do that by thinking about how this particular program works.

To calculate a particular value `Fibonacci(n)`, the program first recursively calculates `Fibonacci(n - 1)`, `Fibonacci(n - 2)`, ..., `Fibonacci(2)`. It then looks up `Fibonacci(1)` and `Fibonacci(0)` in the `FibonacciValues` array.

As each recursive call finishes, it saves its value in the `FibonacciValues` array so that it can be used by calls to the algorithm higher up the call stack. To make this work, the algorithm saves new values into the array in increasing order. As the recursive calls finish, they save `Fibonacci(2)`, `Fibonacci(3)`, . . . , `Fibonacci(n)` in the array.

Knowing this, you can remove the recursion by making the algorithm follow similar steps to create the Fibonacci values in increasing order. Instead of starting with the highest-level call to calculate `Fibonacci(n)`, you use a bottom-up approach. You start by calculating the smallest value `Fibonacci(2)` and work your way up.

The following pseudocode shows this approach:

```
// Return the nth Fibonacci number.
Integer: Fibonacci(Integer: n)
    If (n > MaxN) Then
        // Calculate values between Fibonacci(MaxN) and Fibonacci(n).
        For i = MaxN + 1 To n
            FibonacciValues[i] = Fibonacci(i - 1) + Fibonacci(i - 2)
        Next i

        // Update MaxN.
        MaxN = n
    End If

    // Return the calculated value.
    Return FibonacciValues[n]
End Fibonacci
```

This version of the algorithm starts by precalculating all the Fibonacci values up to the one it needs. It then returns that value.

General Recursion Removal

The previous sections explained how you can remove tail recursion, how dynamic programming lets you save previously calculated values, and how you can remove recursion from the Fibonacci algorithm, but they didn't give a general

algorithm for removing recursion in other situations. For example, the Hilbert curve algorithm is multiply recursive, so you can't use tail recursion removal on it. You might be able to work on it long enough to come up with a nonrecursive version, but that would be hard.

A more general way to remove recursion is to mimic what the program does when it performs recursion. Before making a recursive call, the program stores information about its current state on the stack. Then, when the recursive call returns, the program pops the saved information off of the stack so that it can resume execution where it left off.

To mimic this behavior, divide the algorithm into sections that come before each recursive call, and name them 1, 2, 3, and so forth.

Next, create a variable named `section` that indicates which section the algorithm should execute next. Set this variable to 1 initially so that the algorithm starts with the first section of code.

Create a `while` loop that executes as long as `section` is greater than 0.

Now move all the algorithm's code inside the `while` loop and place it inside a series of `If-Else` statements. Make each `If` statement compare the variable `section` to a section number and execute the corresponding code if they match. (You can use a `Switch` or `Select Case` statement instead of a series of `If-Else` statements if that makes sense for your programming language.)

When the algorithm enters a section of code, increment the variable `section` so that the algorithm knows which section to execute the next time it passes through the loop.

When the algorithm would normally call itself recursively, push all of the parameters' current values onto stacks. Also, push `section` onto a stack so that the algorithm will know which section to execute when it returns from the simulated recursion. Update any parameters that should be used by the simulated recursion. Finally, set `section = 1` to begin the simulated recursive call at the first section of code.

The following pseudocode shows the original Hilbert curve algorithm presented earlier in this chapter broken into sections after each recursion:

```
Hilbert(Integer: depth, Float: dx, Float: dy)
    // Section 1.
    If (depth > 0) Then Hilbert(depth - 1, dy, dx)

    // Section 2.
    DrawRelative(dx, dy)
    If (depth > 0) Then Hilbert(depth - 1, dx, dy)

    // Section 3.
    DrawRelative(dy, dx)
    If (depth > 0) Then Hilbert(depth - 1, dx, dy)
```

```

// Section 4.
DrawRelative(-dx, -dy)
If (depth > 0) Then Hilbert(depth - 1, -dy, -dx)

// Section 5.
End Hilbert

```

The following pseudocode shows this code translated into a nonrecursive version:

```

// Draw the Hilbert curve.
Hilbert(Integer: depth, Float: dx, Float: dy)
    // Make stacks to store information before recursion.
    Stack<Integer> sections = new Stack<int>();
    Stack<Integer> depths = new Stack<int>();
    Stack<Float> dxs = new Stack<float>();
    Stack<Float> dys = new Stack<float>();

    // Determine which section of code to execute next.
    Integer: section = 1

    While (section > 0)
        If (section == 1) Then
            section = section + 1
            If (depth > 0) Then
                sections.Push(section)
                depths.Push(depth)
                dxs.Push(dx)
                dys.Push(dy)
                // Hilbert(depth - 1, gr, dy, dx)
                depth = depth - 1
                float temp = dx
                dx = dy
                dy = temp
                section = 1
            End If
        Else If (section == 2) Then
            DrawRelative(gr, dx, dy)
            section = section + 1
            If (depth > 0) Then
                sections.Push(section)
                depths.Push(depth)
                dxs.Push(dx)
                dys.Push(dy)
                // Hilbert(depth - 1, gr, dx, dy)
                depth = depth - 1
                section = 1
            End If
        Else If (section == 3) Then
            DrawRelative(gr, dy, dx)
            section = section + 1
        End If
    End While

```

```
    If (depth > 0) Then
        sections.Push(section)
        depths.Push(depth)
        dxs.Push(dx)
        dys.Push(dy)
        // Hilbert(depth - 1, gr, dx, dy)
        depth = depth - 1
        section = 1
    End If
Else If (section == 4) Then
    DrawRelative(gr, -dx, -dy)
    section = section + 1
    If (depth > 0) Then
        sections.Push(section)
        depths.Push(depth)
        dxs.Push(dx)
        dys.Push(dy)
        // Hilbert(depth - 1, gr, -dy, -dx)
        depth = depth - 1
        float temp = dx
        dx = -dy
        dy = -temp
        section = 1
    End If
Else If (section == 5) Then
    // Return from a recursion.
    // If there's nothing to pop, we're at the top.
    If (sections.Count == 0) Then section = -1
    Else
        // Pop the previous parameters.
        section = sections.Pop()
        depth = depths.Pop()
        dx = dxs.Pop()
        dy = dys.Pop()
    End If
End While
End Hilbert
```

This version is quite a bit longer, because it contains several copies of code to push values onto stacks, update parameters, and pop values back off stacks.

The technique doesn't really help much for the Hilbert curve because a level 8 or 9 curve will fill every pixel on your computer's screen, so there's no reason to go to any greater depth of recursion. It's still a useful technique to know when you want to remove deep levels of recursion.

Summary

Recursion is a powerful technique. Some problems are naturally recursive, and when they are, a recursive algorithm is often much easier to design than a nonrecursive version. For example, recursion makes the Tower of Hanoi puzzle

relatively easy to solve. Recursion also lets you create interesting pictures such as self-similar curves and gaskets with little code.

Recursion lets you implement backtracking algorithms and solve problems in which you need to repeat certain steps an unknown number of times. For example, generating selections or permutations is easy if you know how many items you will need to pick when you write the code. If you don't know beforehand how many items to pick, generating solutions is easier with recursion.

Despite its usefulness, recursion can sometimes cause problems. Using recursion indiscriminately can make a program repeat the same calculation many times, as does the most obvious implementation of the Fibonacci algorithm. Deep levels of recursion can also exhaust the stack space and make a program crash. In cases such as these, you can remove recursion from a program to improve performance.

Aside from these few instances, recursion is an extremely powerful and useful technique. It's particularly useful when you're working with naturally recursive data structures, such as the trees described in the next three chapters.

Exercises

You can find the answers to these exercises in Appendix B. Asterisks indicate particularly difficult problems.

Some of the following exercises require graphic programming. Exactly how you build them depends on your programming environment. They also require graphic programming experience, so they are marked with asterisks to indicate that they are harder than the other problems.

Other programs, such as the eight queens problem and the knight's tour, can be implemented graphically or just with textual output. Start with text output and then implement the programs graphically if you want an extra challenge.

1. Write a program that implements the original recursive Factorial algorithm. Experiment with it to see how large a factorial you can reasonably calculate on your computer.
2. Write a program that implements the original recursive Fibonacci algorithm. Experiment with it to see how large a value you can calculate on your computer in around 10 seconds.
3. Write a program that implements the Tower of Hanoi algorithm. The result should be text that shows the series of moves in the form A-->B where this represents moving the top disk from peg A to peg B. For example, here is the result for moving three disks:

```
A-->B A-->C B-->C A-->B C-->A C-->B A-->B
```

4. *Write a program that solves the Tower of Hanoi puzzle and then displays the moves by graphically drawing disks moving between the pegs. (For hints, see Appendix B.)
5. *Write a program that draws Koch snowflakes. What is the maximum depth beyond which you can't really see any difference in the curve?
6. *In the standard Koch snowflake, the generator's corners are 60-degree angles, but you can use other angles to produce interesting results. Write a program that lets the user specify the angle as a parameter and that produces a result similar to the one shown in Figure 9.23 for 80-degree angles.

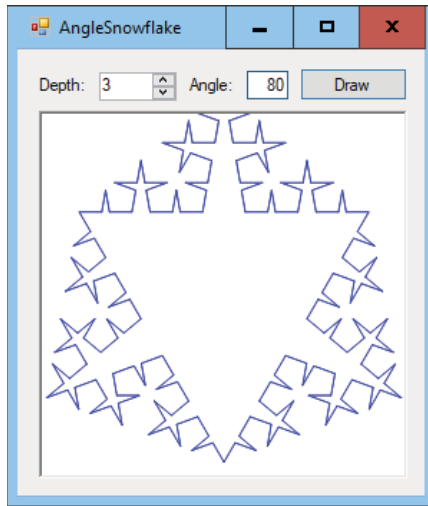


Figure 9.23: Giving the Koch snowflake's generator 80-degree turns creates a spiky result.

7. *Write a program that draws Hilbert curves. (For a hint about how to set dx , see Appendix B.)
8. Write pseudocode for the algorithms that draw the Sierpiński curve pieces down, left, and up.
9. *Write a program that draws Sierpiński curves. (For a hint about how to set dx , see Appendix B.)
10. Write low-level pseudocode to draw the Sierpiński gasket.
11. Write low-level pseudocode to draw the Sierpiński carpet.
12. *Write a program that solves the eight queens problem. In addition to the solution, display the number of queen positions examined and the time the program takes to solve the problem.

13. *One improvement that you can make to the eight queens problem is to keep track of how many queens can attack a particular position on the board. Then, when you are considering adding a queen to the board, you can ignore any positions where this value isn't 0. Modify the program you wrote for Exercise 12 to use this improvement.
14. *To make another improvement to the eight queens problem, notice that every row on the chessboard must hold a single queen. Modify the program you wrote for Exercise 13 so that each call to the `EightQueens` method searches only the next row for the new queen's position.
15. Compare the number of queen positions examined and the elapsed time for the programs that you wrote for Exercises 12, 13, and 14, respectively.
16. *Write a program that uses backtracking (but not Warnsdorff's heuristic) to solve the knight's tour problem. Let the user specify the board's width and height. What is the size of the smallest square board larger than 1×1 for which a knight's tour is possible? What is the size of the largest board for which the program can find a solution in under 10 seconds?
17. Write a program that solves the knight's tour problem by using Warnsdorff's heuristic. How large a board can the program handle?
18. How are a selection without duplicates and a permutation without duplicates related?
19. Write a program that implements the `SelectKofNwithDuplicates` and `SelectKofNwithoutDuplicates` algorithms.
20. Write a program that implements the `PermuteKofNwithDuplicates` and `PermuteKofNwithoutDuplicates` algorithms.
21. Write a program that implements the nonrecursive Factorial algorithm. What is the largest factorial that you can calculate with your program?
22. Write a program that uses dynamic programming to calculate Fibonacci numbers. What is the largest value that you can calculate with an initially empty table of values?
23. Write a program that implements the nonrecursive, bottom-up Fibonacci algorithm. What is the largest value that you can calculate with this program?
24. The nonrecursive Fibonacci algorithm calculates Fibonacci numbers up to the one it needs and then looks up the value in its array. In fact, the algorithm doesn't really need the array. Instead, it can calculate the smaller Fibonacci values whenever they are needed. This takes a little longer but

avoids the need for a globally available array. Write a program that implements a nonrecursive Fibonacci algorithm that uses this approach. Does this change the values that you can calculate?

25. *Write a program that implements the nonrecursive Hilbert curve algorithm.